



Universiteit
Utrecht



Tiago Bastos

25-09-2026

Versie
2.0

Opdrachtgever – Bedrijfsbegeleider
Universiteit Utrecht – Rik D.T. Janssen

1^e examiner – 2^e examiner
Rik Boss – Wim Wennekes

Modernisatie Ricgraph Explorer

Eindverslag

Inhoud

1.	Managementsamenvatting	4
2.	Inleiding	6
3.	Projectcontext.....	7
3.1	Organisatorische context	7
3.2	De opdracht	7
3.3	Analyse van de kwestie.....	8
3.4	Doelstellingen	9
4.	Onderzoek.....	10
4.1	Hoofdvraag en deelvragen.....	10
4.2	Theoretisch kader	10
4.3	Kwaliteitsaspecten	14
4.4	Methoden en technieken.....	15
5.	Aanpak en realisatie	17
5.1	Gekozen aanpak.....	17
5.2	Onderbouwing van gemaakte keuzes.....	17
5.3	Realisatie van de backend	20
5.4	Realisatie van de frontend	20
5.5	Problemen en uitdagingen	21
6.	Resultaten	24
6.1	Gerealiseerde beroepsproducten.....	24
6.2	Validatie en testresultaten.....	25
6.3	Verantwoording kwaliteitscriteria.....	26
6.4	Beantwoording van de deelvragen.....	27
7.	Conclusie.....	29
8.	Aanbevelingen.....	30
9.	Evaluatie.....	32
9.1	Evaluatie van de procesgang.....	32
9.2	Planning, risico's, en afwijkingen	32
9.3	Ethische afwegingen	33
9.4	Randvoorwaarden en kwaliteitsbewaking	34
10.	Verwijzingen	35

11. Bijlages	37
11.1 Bijlage 1 – TypeScript vs JavaScript.....	37
11.2 Bijlage 2 – React Bootstrap.....	39
11.3 Bijlage 3 – Flask vs FastAPI	43
11.4 Bijlage 4 – Stand van zaken.....	49

1. Managementsamenvatting

Dit verslag beschrijft de afstudeeropdracht die is uitgevoerd binnen de afdeling Waardestroom Onderzoek van Information and Technology Services (ITS) van de Universiteit Utrecht. De opdracht richtte zich op het moderniseren van Ricgraph Explorer, een webapplicatie waarmee gebruikers onderzoeksinformatie uit Ricgraph kunnen doorzoeken en visualiseren.

Aanleiding

Ricgraph Explorer was opgezet als een monolithische Flask-applicatie waarin de gebruikersinterface server-side werd gegenereerd binnen dezelfde modules als de data-toegang en de bedrijfslogica. Hoewel Ricgraph al beschikte over een REST API met een OpenAPI-beschrijving, maakte de interface daar geen gebruik van. Daardoor raakte een wijziging in de interface vrijwel altijd ook de data laag, en was het lastig om alternatieve interfaces te ontwikkelen. Daarnaast retourneerden verschillende API-endpoints grote resultaten sets zonder paginering, wat bij groeiende datasets leidt tot onnodig dataverkeer en langere responstijden.

Doelstelling en hoofdvraag

Het doel was de onderhoudbaarheid, uitbreidbaarheid, en prestaties van Ricgraph Explorer te verbeteren zonder bestaande functionaliteit te verliezen. De hoofdvraag luidde: *Hoe kan Ricgraph Explorer worden gemoderniseerd zodat onderhoudbaarheid, uitbreidbaarheid, en prestaties worden verbeterd zonder bestaande functionaliteit te verliezen?*

Aanpak

Het project is iteratief uitgevoerd, geïnspireerd op Scrum, met wekelijkse voortgangsgesprekken en periodieke sprintreviews met de bedrijfsbegeleider. Voorafgaand aan de realisatie is de bestaande codebase geanalyseerd en is literatuuronderzoek gedaan naar frontendtechnologieën, UI-frameworks, en paginerings technieken. Op basis daarvan is gekozen voor React met TypeScript en React-Bootstrap aan de frontendzijde, en voor cursor-based paginering aan de backendzijde.

Resultaten

Het project levert drie producten op, zoals vastgelegd in het plan van aanpak. Ten eerste ondersteunen de relevante API-endpoints van Ricgraph Explorer cursor-based paginering. Daarvoor is binnen dit project eerst een eigen implementatie ontworpen en gebouwd over negen endpoints, die op de frontend was aangesloten; die is later vervangen door de implementatie van de bedrijfsbegeleider, die de officieel onderhouden versie binnen Ricgraph werd. Ten tweede een zelfstandige React-frontend die het grootste deel van de functionaliteit van de oorspronkelijke Ricgraph Explorer dekt en uitsluitend via REST met de backend communiceert. Ten derde de werkende combinatie van beide, die de oorspronkelijke applicatie grotendeels kan vervangen. Enkele

functies ontbreken nog, deels omdat de REST API ze niet aanbiedt. Tot die beschikbaar zijn, blijft de oorspronkelijke interface naast de nieuwe bestaan. De frontend is modulair opgebouwd: herbruikbare onderdelen zoals de gepagineerde tabel, de tabbladtabel, filtercomponenten, en het histogram worden op meerdere pagina's hergebruikt, en datatoegang is per functioneel gebied gescheiden van weergave.

Later in het traject heeft één inhoudelijke koerswijziging plaatsgevonden. Voor de cursor-based paginering was een eigen implementatie gebouwd en op de frontend aangesloten, toen de bedrijfsbegeleider een eigen implementatie aanleverde die de officieel onderhouden versie binnen Ricgraph zou worden. Er is besloten de eigen implementatie te laten vallen en de aangeleverde versie over te nemen, zodat de frontend aansluit op het contract dat daadwerkelijk wordt onderhouden.

Conclusie

De beide knelpunten uit de analyse zijn aangepakt: de applicatie is opgesplitst in een zelfstandige frontend en een Flask-backend die uitsluitend via de REST API communiceren, en de relevante endpoints zijn voorzien van cursor-based paginering. De verbetering in onderhoudbaarheid en uitbreidbaarheid is kwalitatief onderbouwd aan de hand van de scheiding van verantwoordelijkheden, het expliciete API-contract en de mate van componenthergebruik. Aan het eind van het traject is de volledige applicatie systematisch doorlopen tegen een grote dataset. De zoek- en resultaatpagina's functioneren en de paginering is geverifieerd. Twee onderdelen, de pagina voor samenwerkingen en de Open Science-pagina's, functioneren op een dataset van die omvang niet goed. Of de nieuwe oplossing ook een meetbare prestatieverbetering oplevert, is binnen dit project niet vastgesteld: er zijn geen kwantitatieve metingen uitgevoerd.

Aanbevelingen

Aanbevolen wordt om de prestaties alsnog kwantitatief te meten volgens de in hoofdstuk 8 beschreven opzet, te onderzoeken of een database-cursor binnen Neo4j haalbaar en efficiënter is, de aggregatie op de Open Science-pagina's volledig door de cursor te laten lopen in plaats van af te kappen op 250 resultaten, geautomatiseerde tests toe te voegen, en de bruikbaarheid van de nieuwe frontend te toetsen bij daadwerkelijke gebruikers.

2. Inleiding

Dit verslag beschrijft de werkzaamheden die zijn uitgevoerd tijdens de afstudeerstage bij de Universiteit Utrecht. Het project richtte zich op het moderniseren van Ricgraph Explorer, een webapplicatie voor het doorzoeken en visualiseren van onderzoeksinformatie. De opdracht bestond uit het ontwikkelen van een nieuwe frontend en het verbeteren van de backend om de onderhoudbaarheid, schaalbaarheid, en prestaties van de applicatie te vergroten.

In dit verslag worden de organisatie en projectcontext beschreven, gevolgd door de analyse van de opdracht en de uitgevoerde werkzaamheden. Vervolgens worden de gemaakte keuzes, de gerealiseerde resultaten, en de evaluatie van het project besproken. Tot slot wordt gereflecteerd op het verloop van de opdracht en de behaalde leerdoelen.

Bij het opstellen van dit verslag is gebruikgemaakt van **Claude** (Anthropic, 2026), een generatieve AI-tool. De tool is ingezet als schrijfhulp: voor het beoordelen van de opbouw en leesbaarheid, het signaleren van herhalingen en inconsistenties, en het herformuleren van passages. Daarnaast is de tool gebruikt om feitelijke details in het verslag te controleren tegen de projectdocumentatie en de versiegeschiedenis van de codebase. De inhoudelijke keuzes, en de conclusies in dit verslag zijn van de auteur.

Daarnaast is generatieve AI gebruikt bij het schrijven van code. Het merendeel van de frontend is zelf geschreven, waaronder de basisopzet, de componentstructuur, en de datatoegangslaag. Waar AI is ingezet, ging het om ondersteuning en is de gegenereerde code beoordeeld en aangepast voordat deze werd overgenomen. Voor drie pagina's is de code grotendeels door AI gegenereerd: de twee Open Science Landscape-pagina's en de pagina met het Sankey-diagram voor samenwerkingen. Dit waren de meest complexe pagina's van de applicatie. Voor geen van deze pagina's bestaat een endpoint dat de benodigde gegevens kant-en-klaar teruggeeft, waardoor zowel de aggregatie als de opbouw van de visualisatie in de frontend moet gebeuren. In combinatie met de resterende projecttijd is ervoor gekozen deze pagina's met behulp van AI te realiseren. De gegenereerde code is niet regel voor regel doorgrond, en de juistheid van de getoonde gegevens op deze pagina's is niet volledig geverifieerd.

3. Projectcontext

3.1 Organisatorische context

Deze afstudeeropdracht is uitgevoerd binnen de Universiteit Utrecht, bij de afdeling Waardestroom Onderzoek van Information and Technology Services (ITS). Dit team ondersteunt onderzoekers en onderzoeksprocessen binnen de universiteit door het ontwikkelen en beheren van verschillende digitale onderzoeksvoorzieningen.

Eén van deze voorzieningen is Ricgraph (Research in Context Graph), een open-source applicatie die onderzoeksinformatie uit verschillende bronsystemen samenbrengt in een centrale graafdatabase. Hierdoor kunnen relaties tussen onderzoekers, organisaties, publicaties, datasets, en andere onderzoeksobjecten inzichtelijk worden gemaakt en geanalyseerd.

Binnen dit project vervulde de student de rol van software developer.

3.2 De opdracht

De afstudeeropdracht bestond uit het moderniseren van Ricgraph Explorer, de webapplicatie waarmee gebruikers gegevens uit Ricgraph kunnen doorzoeken en visualiseren. De opdracht omvatte zowel frontend- als backendwerkzaamheden.

Aan de frontenzijde was het doel om een nieuwe gebruikersinterface te ontwikkelen met behulp van React en TypeScript. Deze frontend moest de bestaande functionaliteit van Ricgraph Explorer ondersteunen en gebruikmaken van modulaire, herbruikbare componenten. Aan de backenzijde lag de focus op het verbeteren van de prestaties door het implementeren van cursor-based paginering op API-endpoints die grote hoeveelheden data retourneren.

De opdrachtgever had vooraf een gewenste technologieselectie vastgesteld. Tijdens het project is onderzocht of alternatieve technologieën voordelen zouden kunnen bieden. Op basis van een vergelijking van verschillende opties is geconcludeerd dat de voorgestelde technologieën voldoende geschikt waren voor de opdracht en dat een overstap naar een andere stack geen duidelijke meerwaarde zou opleveren binnen de beschikbare projectduur.

Zo is onder meer onderzocht of een migratie van Flask naar FastAPI voordelen zou bieden (zie bijlage 3). FastAPI scoort beter op inputvalidatie, automatische documentatie, en gelijktijdige verwerking, maar een migratie van de backend viel expliciet buiten de scope van deze opdracht en is daarom niet uitgevoerd.



Figuur 1 Organogram

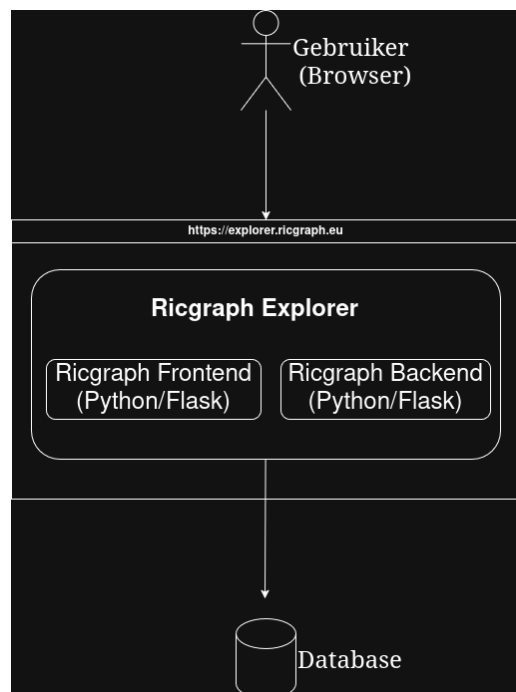
De verwachte eindresultaten bestonden uit een backend waarin relevante API-endpoints gebruikmaken van cursor-based paginering en een functionele React-frontend die de bestaande Ricgraph Explorer kan vervangen. Daarnaast werd verwacht dat de frontend modulair zou zijn opgebouwd, zodat componenten eenvoudig herbruikbaar zijn voor toekomstige uitbreidingen of alternatieve gebruikersinterfaces.

In de volgende hoofdstukken wordt toegelicht welke onderdelen zijn gerealiseerd, welke keuzes daarbij zijn gemaakt en welke werkzaamheden nog openstaan. Omdat binnen een bestaand project is gewerkt, geeft paragraaf 6.1 een overzicht van welke onderdelen zelf zijn ontwikkeld, en welke al bestonden of zijn overgenomen.

3.3 Analyse van de kwestie

Ricgraph Explorer is de webapplicatie waarmee gebruikers onderzoeksinformatie uit Ricgraph kunnen doorzoeken en visualiseren. Bij de start van het project was deze applicatie opgezet als een monolithische Flask-applicatie, waarbij frontend- en backendfunctionaliteit sterk met elkaar verweven waren. Deze architectuur zorgde voor verschillende knelpunten.

Voor de Universiteit Utrecht vormt dit een risico voor de verdere ontwikkeling van Ricgraph. Naarmate meer data wordt toegevoegd en nieuwe wensen ontstaan, neemt de technische schuld toe en worden aanpassingen steeds lastiger door te voeren. Daarom was behoefte aan een modernere architectuur die beter aansluit bij hedendaagse webapplicaties.



Figuur 2 Huidige architectuur

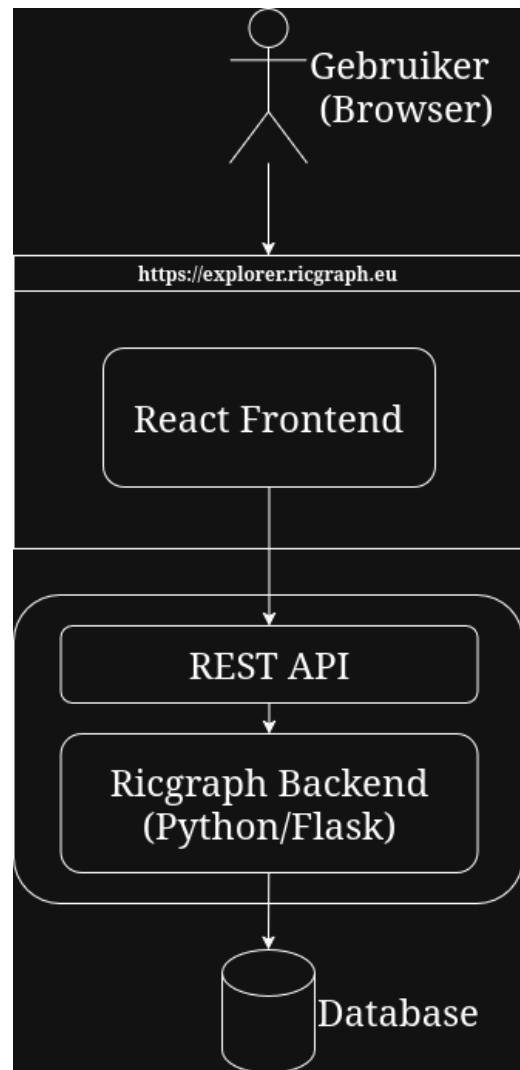
Ricgraph beschikte al sinds 2024 over een REST API met een bijbehorende OpenAPI-beschrijving, die door de opdrachtgever wordt onderhouden. De gebruikersinterface van Ricgraph Explorer maakte hier echter geen gebruik van. De pagina's werden server-side (backend) gegenereerd binnen dezelfde Flask-applicatie, in modules waarin presentatie, datatoegang, en bedrijfslogica door elkaar liepen. Een wijziging in de interface raakte daardoor vrijwel altijd ook andere lagen en omgekeerd, en het was lastig om alternatieve of aanvullende interfaces te ontwikkelen. Daarnaast retourneerden verschillende API-endpoints grote resultatensets zonder paginering, wat kon leiden tot onnodig dataverkeer en verminderde prestaties bij grotere datasets.

De beperking lag dus niet in het ontbreken van een API, maar in het feit dat de bestaande interface er niet op aansloot en dat de API nog geen paginering ondersteunde. Deze constatering wijkt af van de aanname in het afstudeervoorstel, waarin het

ontwerp en de implementatie van een REST API als onderdeel van de opdracht waren opgenomen. De scope is hierop tijdens het project bijgesteld: het accent kwam te liggen op het uitbreiden van de bestaande API met paginering, en op het bouwen van de frontend als zelfstandige client op dat bestaande contract. Deze bijstelling is ook toegelicht in hoofdstuk 9.

3.4 Doelstellingen

Het primaire doel van deze afstudeeropdracht was het moderniseren van Ricgraph Explorer, zodat de applicatie beter onderhoudbaar, schaalbaar, en uitbreidbaar zou worden. Om dit te bereiken werd beoogd de bestaande monolithische architectuur gedeeltelijk te vervangen door een oplossing waarin frontend en backend duidelijker van elkaar gescheiden zijn. Figuur 3 laat een schets zien van hoe de nieuwe architectuur eruit zou zien. De concrete doelen voor frontend en backend zijn beschreven in 3.2.



Figuur 3 Nieuwe architectuur

4. Onderzoek

4.1 Hoofdvraag en deelvragen

De hoofdvraag luidde:

Hoe kan Ricgraph Explorer worden gemoderniseerd zodat onderhoudbaarheid, uitbreidbaarheid, en prestaties worden verbeterd zonder bestaande functionaliteit te verliezen?

De deelvragen waren:

- Welke onderdelen van de huidige architectuur veroorzaken beperkingen op het gebied van onderhoudbaarheid en uitbreidbaarheid?
- Welke aanpassingen zijn nodig om de bestaande Ricgraph Explorer op te splitsen in een zelfstandige frontend en backend?
- Welke vorm van paginering is het meest geschikt voor de datasets en zoekfunctionaliteiten van Ricgraph?
- Hoe kan worden vastgesteld dat de nieuwe oplossing een verbetering vormt ten opzichte van de bestaande situatie?

4.2 Theoretisch kader

Monolithische architectuur

Een monolithische architectuur is een softwarearchitectuur waarbij verschillende onderdelen van een applicatie, zoals de gebruikersinterface, bedrijfslogica, en datatoegang, binnen één applicatie en codebase worden ontwikkeld en uitgevoerd. Hierdoor zijn alle onderdelen sterk met elkaar verbonden (Awati & Wigmore, 2024).

Een voordeel van een monolithische architectuur is dat de applicatie relatief eenvoudig te ontwikkelen en uit te rollen is, omdat alle functionaliteit zich op één plek bevindt. Een nadeel is echter dat onderhoud en uitbreiding lastiger worden naarmate de applicatie groeit. Wijzigingen in één onderdeel kunnen gevolgen hebben voor andere onderdelen van het systeem, waardoor de complexiteit toeneemt.

Een opdeling van een monoliet is echter niet altijd wenselijk. (Fowler, 2015) stelt dat het vaak verstandiger is met een monoliet te beginnen en pas op te splitsen wanneer de grenzen tussen onderdelen duidelijk zijn, omdat een te vroege opdeling extra complexiteit en verkeerd gekozen scheidslijnen oplevert. Een "modulaire monoliet", waarbij onderdelen binnen één applicatie strikt gescheiden blijven, wordt gezien als een volwaardig alternatief voor een volledige opdeling. Binnen dit project is de opdeling daarom beperkt gehouden tot een scheiding tussen frontend en backend; de backend zelf blijft één Flask-applicatie.

Binnen Ricgraph Explorer was sprake van een monolithische architectuur. De frontend, backend, en overige functionaliteit waren ondergebracht in dezelfde Flask-

applicatie. Dit vormde één van de belangrijkste aanleidingen voor de modernisering van het systeem.

Bij het moderniseren van een bestaand systeem ligt een volledige vervanging in één keer voor de hand: een nieuw systeem bouwen dat precies doet wat het oude deed, maar in modernere technologie. (Fowler, 2024) stelt dat deze aanpak in de praktijk meestal mislukt. Een vervanging kost veel tijd, het bestaande gedrag blijkt lastig volledig te achterhalen, en een groot deel van dat gedrag is eigenlijk niet gewenst. Als alternatief beschrijft hij een geleidelijke modernisering, bekend als de "Strangler Fig"-benadering: nieuwe onderdelen worden naast het bestaande systeem gebouwd, maar los daarvan, en functionaliteit verhuist stap voor stap van de oude naar de nieuwe code. Daarvoor zijn plekken nodig waarlangs een systeem kan worden opgesplitst. Tijdens de overgang bestaan het oude en het nieuwe systeem naast elkaar.

Binnen Ricgraph Explorer was zo'n plek al aanwezig in de vorm van de bestaande REST API. De opdracht was zo afgebakend dat alleen de gebruikersinterface opnieuw werd gebouwd, als zelfstandige applicatie die via deze API communiceert met de bestaande backend; een migratie van die backend viel buiten de scope (zie 3.2). De oorspronkelijke, server-side gegenereerde interface is daarbij blijven bestaan naast de nieuwe frontend. Het uitfasen van die oude interface, de laatste stap van een geleidelijke modernisering, is binnen dit project niet uitgevoerd.

Frontend/backend scheiding

Bij frontend/backend scheiding worden de gebruikersinterface en de serverlogica van elkaar gescheiden. De frontend is verantwoordelijk voor de presentatie van gegevens aan de gebruiker, terwijl de backend verantwoordelijk is voor gegevensverwerking, bedrijfslogica, en communicatie met databases (Back4app Engineering, 2026).

Een belangrijk voordeel van deze scheiding is dat beide onderdelen onafhankelijk van elkaar kunnen worden ontwikkeld en onderhouden. Hierdoor neemt de onderhoudbaarheid toe en wordt het eenvoudiger om nieuwe functionaliteit toe te voegen of alternatieve gebruikersinterfaces te ontwikkelen.

Een scheiding brengt ook nadelen mee. Er ontstaat een netwerkgrens tussen beide onderdelen, met bijbehorende aandachtspunten zoals CORS, versiebeheer van het API-contract, en het gescheiden uitrollen van twee applicaties. Fowler (2015) stelt in dat verband dat het opsplitsen van een systeem pas verstandig is wanneer de bestaande opzet daadwerkelijk knelt, omdat de gedistribueerde variant zelf ook complexiteit toevoegt. Voor dit project wegen die nadelen minder zwaar, omdat frontend en backend door dezelfde ontwikkelaar en binnen hetzelfde project worden onderhouden.

Binnen dit project werd een nieuwe frontend ontwikkeld die losstaat van de bestaande backend. Hierdoor ontstaat een duidelijkere scheiding van verantwoordelijkheden en een meer modulaire architectuur.

REST API

Een Application Programming Interface (API) maakt communicatie tussen verschillende softwaresystemen mogelijk. Een veelgebruikte vorm hiervan is een REST API (Representational State Transfer), waarbij gegevens via HTTP-verzoeken worden uitgewisseld (Gupta, 2025).

Een REST API biedt een gestandaardiseerde manier waarop een frontend gegevens kan opvragen of wijzigen zonder directe toegang te hebben tot de interne werking van de backend. Hierdoor ontstaat een duidelijke scheiding tussen beide systemen.

Omdat de nieuwe frontend binnen dit project onafhankelijk van de backend functioneert, wordt gebruikgemaakt van REST API-endpoints om gegevens uit Ricgraph op te vragen en weer te geven.

Naast REST bestaan alternatieven zoals GraphQL en RPC-gebaseerde API's. Een bekend bezwaar tegen REST is dat een client bij samengestelde gegevens meerdere verzoeken moet doen, of meer gegevens ontvangt dan nodig; GraphQL is daar juist op gericht. Dat bezwaar speelt ook in dit project: bij een paginagrootte van 250 vraagt een resultaten-set van 25.000 items honderd afzonderlijke verzoeken, terwijl de oorspronkelijke interface die verzameling in één keer server-side opbouwde. REST is hier desondanks de logische keuze, omdat het aansluit bij de bestaande, op HTTP gebaseerde opzet van Ricgraph en omdat de opdrachtgever daarvoor al een OpenAPI-beschrijving onderhoudt.

Component-based development

Component-based development is een ontwikkelmethode waarbij een applicatie wordt opgebouwd uit zelfstandige en herbruikbare componenten. Ieder component bevat een afgebakende functionaliteit en kan op meerdere plaatsen binnen een applicatie worden toegepast (Component Based Software Engineering, 2022).

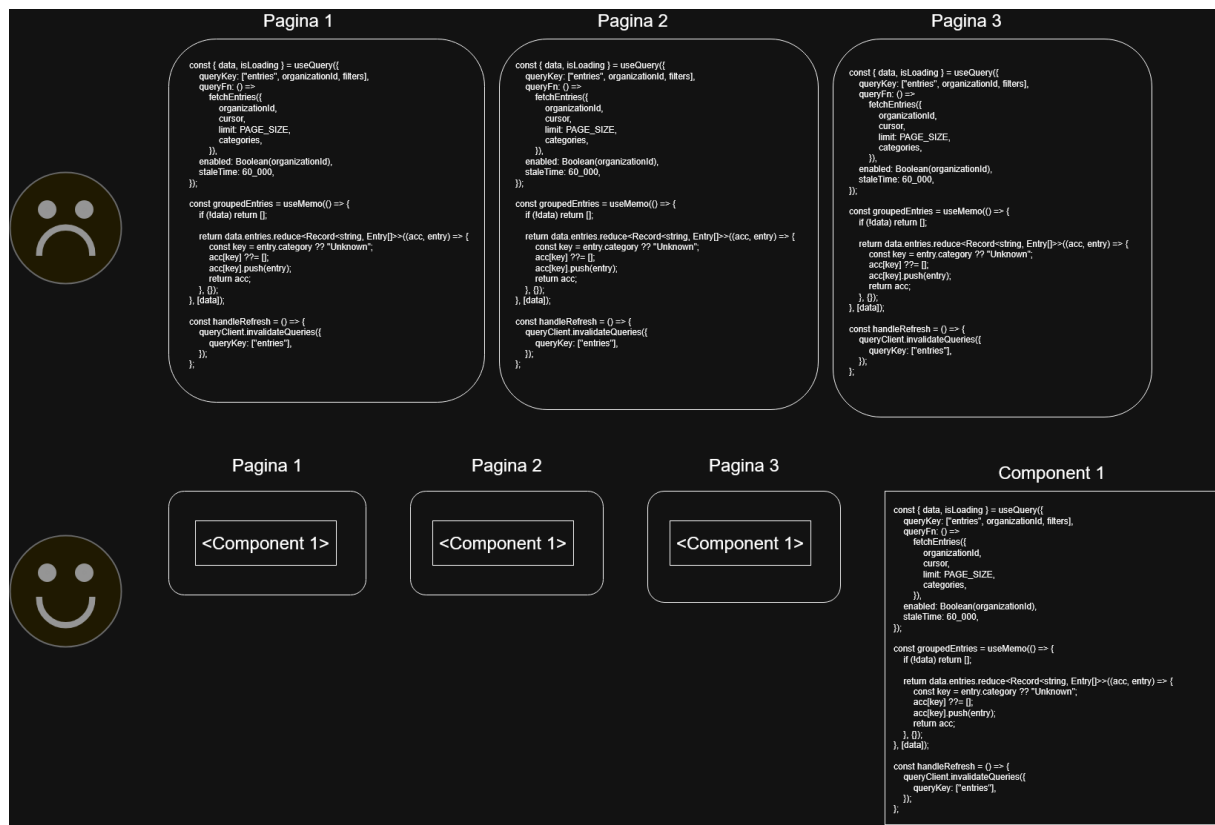
Frameworks zoals React zijn gebaseerd op dit principe. Door gebruik te maken van componenten kan code efficiënter worden beheerd, neemt hergebruik toe, en wordt onderhoud eenvoudiger. Daarnaast zorgt deze aanpak ervoor dat nieuwe functionaliteit eenvoudiger kan worden toegevoegd zonder bestaande onderdelen ingrijpend aan te passen.

Binnen dit project vormde component-based development een belangrijk uitgangspunt bij het ontwikkelen van de nieuwe frontend. Door veelgebruikte onderdelen onder te brengen in herbruikbare componenten wordt de onderhoudbaarheid en uitbreidbaarheid van de applicatie verbeterd.

Figuur 4 is een visuele weergave van hoe componenten kunnen helpen in een project.

Ook hier zijn er grenzen. Verder opsplitsen levert niet vanzelf meer op: het aantal onderdelen groeit en de samenhang wordt minder direct zichtbaar. Omdat het begrip

modulariteit geen natuurlijk eindpunt kent, is niet op voorhand vast te stellen hoe ver opsplitsen zinvol is; deze afweging komt terug in 6.3.



Figuur 4

Pagination

Pagineren is een techniek waarbij grote hoeveelheden gegevens worden opgesplitst in kleinere delen, ook wel pagina's genoemd. In plaats van alle resultaten in één keer terug te sturen, ontvangt de gebruiker slechts een beperkt aantal resultaten per verzoek.

Het toepassen van paginering heeft verschillende voordelen. De hoeveelheid data die via het netwerk wordt verstuurd neemt af, de belasting van de server wordt verminderd en de gebruikersinterface blijft overzichtelijk. Dit wordt vooral belangrijk wanneer gewerkt wordt met grote datasets.

Er bestaan verschillende vormen van paginering. Bij offset-paginering wordt gebruik gemaakt van een offset en een limiet. De database slaat hierbij een bepaald aantal resultaten over voordat nieuwe resultaten worden teruggegeven. Deze techniek is eenvoudig te implementeren, maar kan minder efficiënt worden bij grote datasets.

Cursor-paginering maakt gebruik van een unieke waarde uit het laatst ontvangen resultaat om de volgende resultaten op te vragen. Hierdoor hoeft de database geen grote aantallen records over te slaan en blijft de prestatie beter bij omvangrijke datasets.

Cursor-pagination kent ook nadelen ten opzichte van offset-pagination: een gebruiker kan niet direct naar een willekeurige pagina springen, de implementatie is complexer, en een cursor kan ongeldig worden wanneer de onderliggende gegevens of een server-side buffer wijzigen (QuarkAndCode, 2025). Voor Ricgraph wegen deze nadelen minder zwaar dan het prestatieverlies van offset-pagination bij grote, dynamische datasets (Hartemink, 2026).

Omdat Ricgraph grote hoeveelheden onderzoeksinformatie verwerkt, speelt paginering een belangrijke rol binnen dit project. Een belangrijk onderdeel van de opdracht is daarom het implementeren van cursor-pagination op relevante API-endpoints.

4.3 Kwaliteitsaspecten

De doelstelling van dit project is geformuleerd in termen van drie kwaliteitsaspecten. Ze hangen samen, maar beschrijven verschillende dingen.

Onderhoudbaarheid beschrijft hoe eenvoudig een softwaresysteem kan worden aangepast, uitgebreid, of gerepareerd, zonder dat een wijziging grote gevolgen heeft voor andere onderdelen. Factoren die hieraan bijdragen zijn een duidelijke architectuur, een logische code-indeling, en het vermijden van onnodige afhankelijkheden tussen onderdelen.

Schaalbaarheid beschrijft het vermogen van een systeem om efficiënt te blijven functioneren wanneer de hoeveelheid gebruikers, data, of verzoeken toeneemt. Wanneer een systeem onvoldoende schaalbaar is nemen responstijden toe en verslechtert de gebruikerservaring.

Uitbreidbaarheid beschrijft hoe eenvoudig nieuwe functionaliteit kan worden toegevoegd zonder bestaande functionaliteit ingrijpend aan te passen. De belangrijkste voorwaarde hiervoor is een duidelijke scheiding van verantwoordelijkheden: naarmate onderdelen sterker van elkaar afhankelijk zijn, wordt uitbreiden moeilijker.

Alle drie zijn binnen dit project relevant, en om dezelfde redenen. De bestaande Ricgraph Explorer bracht frontend- en backendfunctionaliteit onder in dezelfde applicatie, waardoor wijzigingen complexer werden, alternatieve interfaces lastig te ontwikkelen waren, en grote resultatensets ongepagineerd werden verwerkt. Een architectuur met een zelfstandige frontend, een REST API als expliciet contract, herbruikbare componenten, en paginering op de zware endpoints, adresseert alle drie de aspecten tegelijk.

De drie aspecten sluiten aan bij het softwarekwaliteitsmodel ISO/IEC 25010, dat onder meer onderhoudbaarheid ("maintainability", met daaronder aanpasbaarheid) en prestatie-efficiëntie ("performance efficiency", met daaronder capaciteit) beschrijft als productkwaliteitskenmerken (ISO/IEC 25010). De in hoofdstuk 6 gehanteerde kwaliteitscriteria: functionele geschiktheid, bruikbaarheid, betrouwbaarheid, en efficiëntie, zijn eveneens op dit model gebaseerd.

4.4 Methoden en technieken

Tijdens het afstudeerproject zijn verschillende onderzoeksmethoden, ontwikkelmethoden, en validatietechnieken toegepast. Deze methoden hadden als doel om de bestaande situatie te analyseren, ontwerpkeuzes te onderbouwen, de nieuwe architectuur te realiseren, en de kwaliteit van het eindresultaat te beoordelen.

Literatuuronderzoek

Het literatuuronderzoek bestond uit twee delen, elk met een eigen doel.

Het eerste deel vond plaats in de eerste weken van het project, voorafgaand aan de implementatie, en diende om de technologiekeuzes te onderbouwen. Daarvoor zijn drie vergelijkingen gemaakt: TypeScript ten opzichte van JavaScript, React-Bootstrap ten opzichte van alternatieve UI-frameworks, en Flask ten opzichte van FastAPI. De uitkomsten zijn gepresenteerd en in een daaropvolgende sprintreview met de bedrijfsbegeleider besproken en akkoord bevonden. Ze zijn samengevat in 3.2 en 5.2, en volledig uitgewerkt in bijlagen 1 tot en met 3. Aansluitend is kort onderzoek gedaan naar paginerings technieken voor de backend, waarbij gebruik is gemaakt van een eerder, intern onderzoek naar paginering binnen de Ricgraph REST API, gedaan door een voormalige stagiair die ook aan Ricgraph heeft gewerkt (Hartemink, 2026).

Het tweede deel diende om het theoretisch kader (4.2) en de kwaliteitsaspecten (4.3) te onderbouwen: de begrippen en modellen waarmee de bestaande situatie is geanalyseerd en de nieuwe oplossing is beoordeeld.

Omdat het onderzoek gericht was op concrete ontwerp- en technologiekeuzes, zijn vooral praktijkgerichte bronnen gebruikt: officiële documentatie van de betreffende technologieën, het softwarekwaliteitsmodel ISO/IEC 25010, twee publicaties van Martin Fowler over softwarearchitectuur, technische vergelijkingsartikelen, en kwantitatieve bronnen zoals de State of JS-enquête, gebruiksstatistieken van W3Techs, en de benchmarks van TechEmpower.

De technologievergelijkingen dragen bij aan deelvraag 2 en het onderzoek naar paginerings technieken aan deelvraag 3. Het theoretisch kader levert de begrippen voor de analyse van de huidige architectuur (deelvraag 1) en voor de opsplitsing in frontend en backend (deelvraag 2). Het kwaliteitsmodel ISO/IEC 25010 vormt de basis voor het beoordelen van de verbetering (deelvraag 4).

Analyse van de bestaande architectuur

Om de huidige situatie goed te begrijpen is de bestaande Ricgraph Explorer geanalyseerd. Hierbij is de bestaande codebase onderzocht om inzicht te krijgen in de architectuur, de onderlinge afhankelijkheden tussen frontend en backend en de manier waarop gegevens werden verwerkt en weergegeven.

Daarnaast is onderzocht welke onderdelen zonder grote wijzigingen konden worden hergebruikt en welke onderdelen aangepast moesten worden om een duidelijke

scheiding tussen frontend en backend mogelijk te maken. De resultaten van deze analyse vormden de basis voor het uiteindelijke ontwerp van de nieuwe architectuur.

Iteratieve softwareontwikkeling

De realisatie van de oplossing vond iteratief plaats. Functionaliteit werd in kleine stappen ontwikkeld, getest, en vervolgens uitgebreid. Hierdoor konden nieuwe inzichten gedurende het project direct worden verwerkt en konden ontwerpkeuzes waar nodig worden aangepast.

De keuze voor een iteratieve, incrementele werkwijze past bij een opdracht waarin de eisen en de complexiteit van de bestaande codebase vooraf niet volledig te overzien waren. Door in korte cycli te ontwikkelen en na elke cyclus feedback op te halen, konden nieuwe inzichten direct worden verwerkt.

De werkwijze was geïnspireerd op Scrum en kende twee vaste overlegvormen. Wekelijks vond op maandag een voortgangsgesprek plaats met de bedrijfsbegeleider, waarin de stand van zaken, prioriteiten, en openstaande vragen werden besproken. Daarnaast werd tweewekelijks op woensdag een sprintreview gehouden met de bedrijfsbegeleider en een andere betrokkene, waarin het opgeleverde werk werd gedemonstreerd en de volgende sprint werd bepaald. De feedback uit beide overleggen is gebruikt om prioriteiten bij te stellen en de verdere ontwikkeling te sturen.

Softwareontwikkeling

Voor de backend is gebruikgemaakt van Python en het bestaande Flask-framework waarop Ricgraph Explorer is gebaseerd. Binnen deze backend zijn bestaande REST-endpoints aangepast en uitgebreid, waarbij cursor-based paginering is geïmplementeerd om grote datasets efficiënter te verwerken.

De frontend is ontwikkeld met React en TypeScript. Voor de gebruikersinterface is gebruikgemaakt van React Bootstrap. Tijdens de ontwikkeling is nadrukkelijk aandacht besteed aan modulariteit door functionaliteit op te splitsen in herbruikbare componenten. Daarnaast is TanStack Query toegepast voor het ophalen, cachen, en synchroniseren van gegevens uit de REST API.

Validatie

De validatie bestond uit twee onderdelen: doorlopende functionele controle tijdens de ontwikkeling, en een systematische eindcontrole van de volledige applicatie aan het eind van het traject. De opzet en resultaten van die eindcontrole staan in hoofdstuk 6 ("Validatie en testresultaten"). Er zijn geen kwantitatieve prestatiemetingen uitgevoerd; de reden daarvoor en de gevolgen voor deelvraag 4 worden toegelicht in hoofdstuk 6 en 7.

5. Aanpak en realisatie

5.1 Gekozen aanpak

Het project is uitgevoerd volgens een iteratieve werkwijze. Eerst is de bestaande Ricgraph Explorer geanalyseerd om inzicht te krijgen in de architectuur en de belangrijkste technische knelpunten, en is er onderzoek uitgevoerd naar mogelijke oplossingsrichtingen, waaronder frontendtechnologieën en paginerings technieken.

Op basis van deze bevindingen is gestart met de realisatie van de oplossing. Hierbij zijn backend- en frontendfunctionaliteit stapsgewijs ontwikkeld, getest, en verbeterd. Gedurende het project vonden regelmatig overleggen plaats met de bedrijfsbegeleider en werden sprintreviews gebruikt om voortgang te bespreken en feedback te verzamelen.

Door deze aanpak konden nieuwe inzichten direct worden meegenomen in de verdere ontwikkeling van de applicatie.

5.2 Onderbouwing van gemaakte keuzes

Tijdens het project zijn verschillende technische keuzes gemaakt of overgenomen. Een deel van deze keuzes was vooraf al voorgesteld door de opdrachtgever, terwijl andere keuzes gedurende het project zijn ontstaan.

Kort samengevat:

- React en React-Bootstrap
 - Frontend framework en UI-library, aanbevolen door de opdrachtgever, onderbouwd door vergelijking met alternatieven
- TypeScript
 - In plaats van JavaScript, voor statische typecontrole bij een project van deze omvang
- Cursor-paginerings
 - Gebaseerd op eerder onderzoek naar paginerings technieken. Eerst zelf ontwikkeld, later teruggedraaid en vervangen door de overgenomen upstream-implementatie
- TanStack Query
 - Niet vooraf gepland. Tijdens de ontwikkeling toegevoegd als poging om een bug op te lossen, aangehouden vanwege de meerwaarde

De keuzes worden hieronder toegelicht.

React en React-Bootstrap

React lag als frontendframework vast op verzoek van de opdrachtgever. De keuze die binnen dit project is gemaakt betreft het UI-framework dat daarbovenop wordt gebruikt.

React zelf richt zich voornamelijk op componentstructuur en state-management, niet op styling. Standaard HTML-elementen binnen React hebben weinig visuele aantrekkingskracht, waardoor in de praktijk vrijwel altijd een UI-framework wordt toegevoegd. React-Bootstrap combineert React met Bootstrap door alle Bootstrap-componenten te herimplementeren als native React-componenten.

Als alternatieven zijn onder andere Tailwind CSS, Material UI, Foundation, en Chakra UI onderzocht. Tailwind biedt meer ontwerp vrijheid via een utility-first aanpak, maar vereist meer handmatig stylingwerk en maakt JSX-code minder overzichtelijk door het grote aantal classes. Material UI biedt visueel aantrekkelijke, kant-en-klare componenten, maar legt het ontwerp sterk vast binnen de Material Design-stijl. Foundation biedt minder componenten dan Bootstrap en wordt minder actief onderhouden. Chakra UI is toegankelijk en flexibel, maar wijkt qua stijl duidelijk af van Bootstrap. (Zie bijlage 2)

Op basis van deze vergelijking is besloten om React-Bootstrap aan te houden: het sluit goed aan bij React, vermijdt de nadelen van directe Bootstrap-integratie, en biedt voldoende kant-en-klare componenten om snel een consistente interface te ontwikkelen, zonder de steilere leercurve of striktere ontwerpfilosofie van de onderzochte alternatieven.

TypeScript

Voor de frontend is gekozen voor TypeScript in plaats van JavaScript. JavaScript wordt weliswaar door circa 98,9% van alle websites gebruikt (Usage statistics of JavaScript as client-side programming language on websites, sd), maar het ontbreken van statische types leidt bij grotere projecten tot fouten die pas tijdens runtime zichtbaar worden. TypeScript, sinds 2012 ontwikkeld door Microsoft als superset van JavaScript, lost dit op door statische typecontrole toe te voegen, terwijl bestaande JavaScript-code geldig blijft waardoor de overstap daardoor laagdrempelig is.

Uit onderzoek van (JavaScript Flavors, 2020) blijkt dat TypeScript breed gedragen wordt binnen de ontwikkelaarsgemeenschap: 78% van de respondenten gaf aan de taal opnieuw te gebruiken, en 93% was tevreden over de ervaring. Gezien de relatieve omvang van de nieuwe frontend binnen dit project, en de voordelen op het gebied van foutdetectie tijdens ontwikkeling en editor-ondersteuning, is TypeScript als een betere keuze beoordeeld.

Cursor-pagineren

Voor de backend is gekozen voor cursor-based pagination, gebaseerd op eerder onderzoek van (Hartemink, 2026), waarin cursor-based pagination als meest geschikte aanpak naar voren kwam voor grote, dynamische datasets binnen een Unicorn-omgeving met meerdere workers.

De oorspronkelijke opzet was om de cursor op databaseniveau te implementeren: bij elke request zou de Cypher-query zelf, via een WHERE-clausule op basis van de laatst opgehaalde node, de resultaten filteren en sorteren. Tijdens een codebespreking met

de bedrijfsbegeleider bleek deze aanpak echter minder efficiënt dan verwacht. Omdat er geen index op het gebruikte sorteerveld aanwezig was, moest bij elke pagina de volledige resterende resultaten set opnieuw worden opgehaald en gesorteerd voordat de eerstvolgende pagina kon worden bepaald. Bij grote datasets betekende dit dat elke volgende aanvraag opnieuw een omvangrijke deelverzameling moest doorzoeken en sorteren, wat gunstig is voor het geheugengebruik maar tot een onevenredig hoge CPU-belasting leidt naarmate de dataset groeit. Deze afweging is niet empirisch getest binnen dit project, maar vormde wel de directe aanleiding om de aanpak te herzien.

Het daadwerkelijk efficiënt maken van een databasecursor bleek geen triviale opgave: naast het ontbreken van een geschikte index moesten verschillende Cypher-aanroepen elk een eigen, complexere cursor krijgen. Gezien de beschikbare projecttijd en het feit dat de opdracht primair gericht was op de frontend/backend-scheiding, is in overleg met de bedrijfsbegeleider besloten dit onderzoek niet verder uit te diepen binnen de scope van deze stage.

Kort voor zijn vakantie leverde de bedrijfsbegeleider een eigen implementatie van cursor-based paginering aan, met het voorstel deze over te nemen. Deze was op dat moment nog niet in Ricgraph opgenomen; dat stond gepland voor na zijn vakantie. Conceptueel leek de implementatie sterk op de eigen implementatie: ook hier wordt de volledige resultaten set per zoekopdracht eenmalig opgehaald en gecachet, met een cursor die door die cache navigeert. Het API-contract week wel af (zie 6.1).

Omdat deze implementatie na de vakantie de officieel onderhouden versie binnen Ricgraph zou worden, is in overleg besloten de eigen implementatie te laten vallen en die van de bedrijfsbegeleider over te nemen. De overweging: de nieuwe frontend moet aansluiten op het contract dat de opdrachtgever daadwerkelijk onderhoudt, zodat de frontend bij toekomstige updates van Ricgraph niet telkens hoeft te worden aangepast aan een zelfgebouwde variant. De eigen backend-implementatie is hierop volledig teruggedraaid naar de upstream-versie: het komt overeen met de officiële Ricgraph implementatie. De frontend is aangepast aan het nieuwe cursor-contract.

TanStack Query

TanStack Query is, in tegenstelling tot React, TypeScript, en cursor-pagination, niet vooraf gepland, maar gedurende de ontwikkeling toegevoegd. De aanleiding was een hardnekkige bug waarbij bepaalde pagina's of gegevensverzoeken bleven hangen tijdens het ophalen van data (zie "Problemen en uitdagingen"). Als mogelijke oplossing is TanStack Query geïntroduceerd, in de veronderstelling dat de library het datamanagement rondom deze requests zou kunnen stabiliseren.

Achteraf bleek TanStack Query de hangende requests niet op te lossen; de daadwerkelijke oorzaak lag, zoals later bleek, bij twee afzonderlijke problemen elders in de code. TanStack Query is echter aangehouden, omdat tijdens het gebruik duidelijk werd dat de bibliotheek wel degelijk waarde toevoegde: laadstatussen, foutafhandeling, en caching van API-data konden hierdoor consistent en met aanzienlijk minder

handmatige code worden afgehandeld, in plaats van dit per component opnieuw te implementeren.

Deze beslissing was daarmee niet vooraf onderbouwd op basis van literatuuronderzoek, maar het resultaat van een pragmatische afweging tijdens de ontwikkeling: een oplossing die niet het beoogde probleem oploste, maar op basis van de opgedane ervaring alsnog een verbetering bleek voor de kwaliteit en onderhoudbaarheid van de frontend.

5.3 Realisatie van de backend

Het backendwerk binnen dit project bestond uitsluitend uit het ontwerpen en bouwen van een eigen implementatie van cursor-based paginering in de bestaande Python/Flask-applicatie van Ricgraph Explorer. Deze implementatie is later vervangen door die van de bedrijfsbegeleider (zie 5.2). De applicatiecode van de opgeleverde backend is daardoor gelijk aan die in de officiële Ricgraph-codebase. Deze paragraaf beschrijft die eerste eigen implementatie.

Er was dus eerst een eigen, eenvoudigere implementatie gebouwd: per zoekopdracht wordt met één Cypher-query de volledige resultaten-set opgehaald en gecacheerd. Paginering vindt vervolgens plaats over deze gecacheerde lijst, waarbij de cursor een positie in die lijst representeert. De dure database-aanroep wordt zo per zoekopdracht slechts eenmaal uitgevoerd, terwijl navigatie tussen pagina's daarna vrijwel kosteloos is. Deze implementatie besloeg negen API-endpoints, een aangepaste cachinglaag en de bijbehorende aanpassingen aan de OpenAPI-beschrijving, en was aangesloten op de frontend.

De implementatie van paginering bleek niet voor iedere endpoint op dezelfde manier toepasbaar. De structuur van de bestaande API verschilde per endpoint; sommige endpoints retourneerden één lijst met resultaten, terwijl andere endpoints meerdere datasets binnen één response combineerden. Hierdoor moest per endpoint worden onderzocht welke aanpassingen noodzakelijk waren om cursor-based paginering op een consistente manier te kunnen implementeren.

Naast de API-endpoints zijn ook onderliggende functies aangepast om de nieuwe pagineringsmethode te ondersteunen. Hierbij is rekening gehouden met de bestaande cachingmechanismen en de compatibiliteit met reeds aanwezige functionaliteit binnen Ricgraph: tijdens de implementatie is steeds gecontroleerd of bestaande functionaliteit behouden bleef en bestaande gebruikersscenario's correct bleven functioneren.

5.4 Realisatie van de frontend

Voor de frontend is vanaf nul een nieuwe React-applicatie ontwikkeld met React-Bootstrap en TypeScript. Het doel was om de bestaande pagina's van Ricgraph Explorer zo nauwkeurig mogelijk na te bouwen binnen een moderne frontendomgeving.

De frontend is opgebouwd uit een modulaire componentstructuur. Herbruikbare onderdelen zijn ondergebracht in generieke componenten, waaronder een gepagineerde

tabel, een eenvoudige tabel, een tabbladtabel, filtercomponenten, een histogram component, losse zoekveld-componenten, etc.. Datatoegang is gescheiden van weergave: per functioneel gebied bestaat er een contractbestand met de request- en response-typen en een bestand met de bijbehorende ophaalfuncties. Voor het ophalen, cachen, en synchroniseren van API-gegevens wordt TanStack Query gebruikt, met één gedeelde configuratie en centrale foutafhandeling. De ophaallaag werpt bij netwerkfouten en niet-succesvolle antwoorden een fout op in plaats van een lege waarde terug te geven, waardoor onderscheid mogelijk is tussen een onbereikbare backend en een zoekopdracht zonder resultaten. Op dat onderscheid is een statusbanner gebouwd die de gebruiker waarschuwt wanneer de backend niet bereikbaar is. Paginering wordt afgehandeld door herbruikbare hooks. Alle opgehaalde pagina's worden bewaard en samengevoegd, en de tabel toont daaruit steeds één pagina. Bij terugbladeren is daarvoor geen nieuw verzoek naar de backend nodig, en tellingen en histogrammen hebben betrekking op alle tot dan toe geladen resultaten in plaats van alleen op de getoonde pagina.

De gerealiseerde frontend omvat de volgende pagina's:

- Homepage, waarop statistieken dynamisch uit de API worden opgehaald
- Zoekpagina's voor broad search, advanced search, zoeken op persoon, en zoeken op organisatie
- Zoekresultatenpagina, met de gepagineerde resultatentabel
- Keuzepagina, waarop de gebruiker na een zoekopdracht bepaalt welke informatie wordt getoond, zowel voor personen als voor organisaties
- Resultaatpagina's voor personen: alle informatie, identiteiten, onderzoeksoutput, organisaties, gedeelde onderzoeksoutput, samenwerkende organisaties, en verrijking
- Resultaatpagina's voor organisaties: alle informatie, gerelateerde personen, en informatie over personen binnen een organisatie
- Pagina voor overige zoekresultaten
- Pagina's rond samenwerkingen tussen organisaties, met een Sankey-diagram
- Open Science Landscape-pagina's

Onbekende routes worden naar de homepage geleid.

Naast de pagina's zijn enkele gedeelde mechanismen ontwikkeld. Een navigatielaag bepaalt op basis van de categorie van een aangeklikte node naar welke bestemmingspagina wordt genavigeerd, en hergebruikt de al opgehaalde gegevens uit de cache zodat de doelpagina direct kan renderen.

5.5 Problemen en uitdagingen

Tijdens het project zijn verschillende technische uitdagingen naar voren gekomen. Een belangrijk inzicht was dat de complexiteit van de bestaande Ricgraph Explorer vooraf werd onderschat. Hoewel de opdracht in eerste instantie voornamelijk leek te bestaan uit het ontwikkelen van een nieuwe frontend en het implementeren van cursor-based

paginering, bleek al snel dat beide onderdelen sterk afhankelijk waren van de bestaande architectuur. Hierdoor was eerst een uitgebreide analyse van de bestaande codebase noodzakelijk voordat nieuwe functionaliteit veilig kon worden toegevoegd, en dan nog bleek de opdracht nogal uitdagend te zijn.

Een van de grootste technische uitdagingen was het implementeren van een cursor-based paginering in de backend: niet elke endpoint kon op dezelfde manier worden aangepast, en ook onderliggende functies en de cachinglaag moesten worden gewijzigd (zie 5.3 voor de technische toelichting).

Ook de ontwikkeling van de frontend bracht verschillende uitdagingen met zich mee. Vanaf het begin is gewerkt met een componentgebaseerde aanpak, waarbij veelgebruikte onderdelen zijn ondergebracht in afzonderlijke React-componenten. Naarmate de applicatie verder groeide en meer pagina's werden ontwikkeld, bleek echter dat deze componenten nog verder konden worden geabstraheerd, en de opdrachtgever gaf bovendien aan een nog hogere mate van modulariteit te willen, zodat componenten op meer plaatsen binnen de applicatie hergebruikt konden worden. Dit leidde ertoe dat een deel van de frontend opnieuw is gestructureerd en componenten verder zijn opgesplitst. Vooral de tabellen en de paginering zijn uiteindelijk generieker opgezet dan in de oorspronkelijke implementatie.

Tijdens de ontwikkeling werd bovendien een hardnekkige bug ontdekt waarbij bepaalde pagina's of gegevensverzoeken soms bleven hangen tijdens het ophalen van gegevens. Als mogelijke oplossing hiervoor is destijds TanStack Query geïntroduceerd. Dit loste het onderliggende probleem niet op; tijdens verder onderzoek bleek dat twee afzonderlijke oorzaken elders in de code vergelijkbare symptomen veroorzaakten, waardoor het opsporen van de bugs bemoeilijkt werd en het oplossen ervan aanzienlijk meer tijd kostte dan vooraf verwacht. Uiteindelijk konden beide onderliggende problemen afzonderlijk worden geïdentificeerd en verholpen, waarna de stabiliteit van de frontend merkbaar verbeterde.

De eerste oorzaak lag in de frontend, waar een queriesleutel een volledig object bevatte in plaats van een stabiele identificatie, waardoor verzoeken zich niet goed lieten hergebruiken. De tweede lag in de cachinglaag van de backend, waar de gebruikte Memcached-client niet thread-safe was, zodat gelijktijdige verzoeken elkaar konden blokkeren. Beide zijn in korte tijd verholpen.

Een aparte uitdaging ontstond doordat de bedrijfsbegeleider halverwege het traject een eigen implementatie van cursor-based paginering aanleverde, nadat er al een eigen implementatie was gebouwd en aangesloten (de afweging hierachter staat in 5.2).

Bij het testen van de overgenomen implementatie kwamen twee problemen in de code aan het licht. Ten eerste geeft de cursor aan het einde van de resultaten geen lege waarde terug, waardoor een client niet kan detecteren dat er geen volgende pagina meer is. De knop "volgende" bleef daardoor actief en laadde lege pagina's. Dit is in eerste instantie in de frontend opgevangen door een pagina die korter is dan de

opgevraagde paginagrootte als laatste pagina te behandelen. Na terugkoppeling aan de bedrijfsbegeleider is de fout snel in Ricgraph hersteld. De frontend is daarop aangepast en herkent de laatste pagina nu aan het ontbreken van een cursor. Ten tweede konden gelijktijdige verzoeken elkaar blokkeren in de gedeelde cachinglaag, wat zich uitte als een pagina of sectie die oneindig bleef laden. Dit tweede probleem is ook teruggekoppeld, en op hetzelfde moment hersteld. Dit probleem is dezelfde thread-safety-kwestie als de tweede oorzaak van de eerder beschreven bug. De eigen oplossing daarvoor verdween bij het terugzetten van de backend naar de upstream-versie, die deze aanpassing nog niet bevatte. Na terugkoppeling is dezelfde wijziging in Ricgraph doorgevoerd.

6. Resultaten

6.1 Gerealiseerde beroepsproducten

Voordat de afzonderlijke beroepsproducten worden beschreven, volgt hier een overzicht van wat binnen dit project nieuw is ontwikkeld, wat is overgenomen, en wat ongewijzigd is gebleven.

Ricgraph is een bestaand, actief onderhouden open-sourceproject, de werkzaamheden binnen deze opdracht vormen daarbinnen een afgebakend deel.

Onderdeel	Status	Toelichting
React-frontend	Nieuw ontwikkeld	Vanaf nul opgezet: alle pagina's, componenten, etc
Cursor-paginerings in de backend	Ontwikkeld & Overgenomen	Eerst zelf geïmplementeerd op negen endpoints, later teruggedraaid en vervangen door de implementatie van de bedrijfsbegeleider, wat de officieel onderhouden versie werd (zie 5.2)
Ricgraph (kern) en graafdatabase	Bestaand	Buiten de scope, ongewijzigd

De oorspronkelijke Ricgraph Explorer web-interface, die in de backend verweven zit, is blijven bestaan naast de nieuwe frontend, en is binnen dit project niet verwijderd.

Cursor-paginerings in de backend

Cursor-based paginerings is beschikbaar op de relevante API-endpoints van Ricgraph Explorer, waaronder de broad search, de advanced search, en de resultaatpagina's voor personen en organisaties. Binnen dit project is hiervoor eerst een eigen implementatie gebouwd, die negen endpoints besloeg en op de frontend was aangesloten. Die is later vervangen door de implementatie van de bedrijfsbegeleider, die de officieel onderhouden versie binnen Ricgraph werd (zie 5.2).

Functionele React-frontend

Er is een nieuwe frontend ontwikkeld met React, TypeScript, en React-Bootstrap, die het grootste deel van de functionaliteit van de oorspronkelijke Ricgraph Explorer biedt. Enkele functies van de oorspronkelijke interface zijn nog niet beschikbaar. Een deel daarvan wordt door de REST API nog niet aangeboden, de rest wordt door de API wel ondersteund, maar is nog niet in de frontend gebouwd. De frontend communiceert uitsluitend via REST API-endpoints met de backend, en maakt gebruik van TanStack

Query voor het ophalen, cachen, en synchroniseren van gegevens. De applicatie is opgebouwd uit herbruikbare, modulaire componenten, met name zichtbaar in de generieke tabel- en pagineringscomponenten die op meerdere pagina's worden hergebruikt, en in de gedeelde hooks en hulpfuncties, waarin logica voor paginering, filtering, en navigatie los van de pagina's is ondergebracht.

Werkende applicatie

De frontend is een zelfstandige applicatie die werkt tegen elke Ricgraph-backend die deze REST API aanbiedt; welke backend dat is, wordt in de configuratie van de frontend bepaald. Tijdens de eindvalidatie is gewerkt tegen de publieke Ricgraph-demo-server. Gebruikers kunnen onderzoeksinformatie doorzoeken en visualiseren via onder andere de broad search, de advanced search, en de pagina's voor personen, organisaties, en overige zoekresultaten. Twee onderdelen functioneren op een grote dataset niet goed (zie 6.2).

6.2 Validatie en testresultaten

Aan het eind van het traject is de volledige applicatie systematisch doorlopen tegen de backend van de Ricgraph Demo server, met een dataset van ongeveer 2.5 miljoen nodes. Elke route in de applicatie is bezocht en gecontroleerd op correcte weergave, op foutmeldingen in de browserconsole, en op behoud van bestaande functionaliteit. Daarnaast is de paginering, als het onderdeel dat het laatst is gewijzigd, gericht getest.

De pagina's voor zoeken, voor personen, voor organisaties, en voor overige resultaten renderen en functioneren op deze dataset, met een browserconsole die vrij bleef van fouten en uitzonderingen. Twee onderdelen functioneren op een dataset van deze omvang echter niet goed.

De pagina "verken samenwerkingen" bevraagt een endpoint dat geen paginering ondersteunt en dat de volledige resultaten set in één antwoord teruggeeft. Op deze dataset duurt dat langer dan de timeout van tien seconden die de frontend hanteert. De frontend beschouwt de backend daarna als onbereikbaar en toont de statusbanner, waardoor de pagina op deze dataset niet bruikbaar is. Op de kleinere dataset waarmee eerder is ontwikkeld en getest, trad dit niet op.

De Open Science-profiel- en dashboardpagina's halen maximaal 250 resultaten op en bieden geen manier om de rest te laden. Organisaties met meer resultaten worden daardoor ondergerapporteerd, en tellingen op deze pagina's wijken af van tellingen voor hetzelfde item elders in de applicatie.

De paginering is specifiek geverifieerd op de zoekresultatenpagina en op de pagina's met informatie van een persoon of organisatie. Daarbij is vastgesteld dat de knop "volgende" correct doorbladert, dat "vorige" de voorgaande pagina uit de cache bedient zonder nieuw verzoek, dat de knop "volgende" op de laatste pagina wordt uitgeschakeld zodra de backend geen cursor meer teruggeeft.

Alle bevindingen, ook de hier niet besproken punten, zijn met een situatieschets en een aanbeveling opgenomen in bijlage 4.

6.3 Verantwoording kwaliteitscriteria

In het plan van aanpak zijn per beroepsproduct kwaliteitscriteria vastgesteld. Hieronder wordt per product toegelicht in hoeverre aan deze criteria is voldaan.

Backend API: paginering aanwezig, en performance-efficiëntie

Aan het criterium van aanwezige paginering is voldaan: cursor-based paginering is geïmplementeerd op de relevante API-endpoints. Het criterium "performance-efficiënt" is echter niet kwantitatief getoetst; er is geen meting uitgevoerd waarin, zowel de huidige overgenomen implementatie, als de implementatie van de student, zijn vergeleken met bijvoorbeeld de oorspronkelijke, niet-gepagineerde API of met een volwaardige databasecursor. De keuze voor de huidige aanpak is onderbouwd op basis van kwalitatieve analyse, maar een concrete performancevergelijking ontbreekt. Dit is dan ook opgenomen als aanbeveling voor vervolgonderzoek.

Functionele React-frontend: frontend moet bestaan en modulair zijn

Aan beide onderdelen van dit criterium is voldaan. Er is een functionerende React-frontend gerealiseerd die het grootste deel van de functionaliteit van de bestaande Ricgraph Explorer biedt (zie 6.1), en deze is opgebouwd uit herbruikbare componenten. De modulariteit is gedurende het project verder doorgevoerd dan in de eerste opzet, met name in de tabel- en pagineringscomponenten, die generiek genoeg zijn gemaakt om op meerdere pagina's te worden hergebruikt.

Bij dit criterium speelt wel dat "modulair" geen eindpunt kent. Vrijwel elk onderdeel kan verder worden opgesplitst, en de vraag is niet of dat kan, maar tot hoever het zinvol is. Gedurende het project is de modulariteit meermaals verder doorgevoerd, ook op onderdelen die al in herbruikbare componenten waren ondergebracht. Omdat het criterium geen maat bevat voor de gewenste mate van modulariteit, is niet objectief vast te stellen wanneer eraan is voldaan.

Werkende applicatie: functionele geschiktheid, bruikbaarheid, betrouwbaarheid, en efficiëntie

De applicatie ondersteunt de belangrijkste zoek- en visualisatiefunctie van de oorspronkelijke Ricgraph Explorer, maar is nog niet functioneel volledig: enkele functies ontbreken (zie 6.1) en twee onderdelen functioneren op een grote dataset niet goed (zie 6.2). Aan functionele geschiktheid is daarmee niet volledig voldaan. Ook de betrouwbaarheid is niet op alle punten geborgd: doordat de aggregatie op de Open Science-pagina's afkapt op 250 resultaten, wijken tellingen daar af van tellingen voor hetzelfde item elders in de applicatie. Efficiëntie is, net als bij de backend API, niet formeel gemeten; wel is de introductie van TanStack Query een voorbeeld van een

maatregel die de efficiëntie van gegevensverwerking aan de clientzijde beoogt te verbeteren. Bruikbaarheid is niet los getoetst, bijvoorbeeld door middel van gebruikers-tests; dit vormt daarmee eveneens een beperking in de verantwoording van dit criterium.

6.4 Beantwoording van de deelvragen

Welke onderdelen van de huidige architectuur veroorzaken beperkingen op het gebied van onderhoudbaarheid en uitbreidbaarheid?

De belangrijkste beperking is dat de gebruikersinterface server-side wordt gegenereerd binnen dezelfde Flask-applicatie als de backend, in modules waarin presentatie, datatoegang en bedrijfslogica door elkaar lopen. Er bestond wel een REST API met een OpenAPI-beschrijving, maar de interface maakte daar geen gebruik van, waardoor er in de praktijk geen contract tussen de lagen lag. Hierdoor raakt een wijziging in de interface vrijwel altijd ook andere lagen en omgekeerd, en is het lastig om alternatieve of aanvullende interfaces te ontwikkelen. Een tweede beperking is het ontbreken van paginering op endpoints die grote resultatensets retourneren.

Welke aanpassingen zijn nodig om de bestaande Ricgraph Explorer op te splitsen in een zelfstandige frontend en backend?

De bestaande REST API moest worden uitgebreid zodat alle voor de interface benodigde functionaliteit erdoor wordt aangeboden, en zodat endpoints met grote resultatensets gepagineerd kunnen worden opgevraagd. De bijbehorende OpenAPI-beschrijving moest daarop worden bijgewerkt. De frontend moest als zelfstandige applicatie worden herbouwd en uitsluitend via die API communiceren, zodat server-side rendering en directe databasetoegang vanuit de interface overbodig worden. In dit project is deze scheiding voor het grootste deel gerealiseerd. Uit de realisatie bleek dat de API nog niet alle functies van de oorspronkelijke interface aanbiedt. Voor een volledige scheiding moet de API op die punten nog worden uitgebreid. De backend blijft één Flask-applicatie. Een verdere opdeling viel buiten de scope en maakte geen deel uit van de opdracht.

Welke vorm van paginering is het meest geschikt voor de datasets en zoekfunctionaliteiten van Ricgraph?

Cursor-based paginering sluit het best aan, omdat de datasets groot en dynamisch zijn en offset-paginering bij grote datasets prestatieverlies oplevert.

Uit de realisatie bleek daarnaast dat niet elke endpoint cursor-pagineerbaar is, of dat het zo ingewikkeld zou zijn, of zoveel tijd zou kosten, dat het buiten de scope zou vallen. Drie endpoints die resultatensets retourneren zijn zonder cursor gebleven: het opvragen van samenwerkende organisaties van een persoon, het verrijken van een persoon, en het verkennen van samenwerkingen tussen organisaties. De eerste twee

combineren meerdere onafhankelijke lijsten binnen één respons, wat niet in een enkelvoudig cursormodel past. Deze zijn benaderd met een begreind maximumaantal resultaten. De derde endpoint levert een geaggregeerde weergave op waarvoor de volledige resultaten set nodig is en waarbij paginering geen betekenis heeft.

Hoe kan worden vastgesteld dat de nieuwe oplossing een verbetering vormt ten opzichte van de bestaande situatie?

Op twee manieren. Kwalitatief: door de architectuur te toetsen aan onderhoudbaarheids- en uitbreidbaarheidscriteria, de scheiding van verantwoordelijkheden, het bestaan van een expliciet API-contract, en de mate van hergebruik van componenten. Deze verbetering is in dit project aannemelijk gemaakt (zie "Verantwoording kwaliteitscriteria"). Kwantitatief: door responstijd en geheugengebruik van vergelijkbare verzoeken te meten bij oplopende datasetgroottes, voor en na de wijziging, en door het aantal database-aanroepen per pagina te vergelijken. Dit laatste is uit de code af te leiden. De overgenomen implementatie haalt per zoekopdracht één batch op en bedient vervolgpagina's uit de cache, in plaats van per pagina een nieuwe query. Maar is niet met metingen onderbouwd. Een opzet voor deze meting is opgenomen in de aanbevelingen.

7. Conclusie

De hoofdvraag van dit afstudeerproject luidde: Hoe kan Ricgraph Explorer worden gemoderniseerd zodat onderhoudbaarheid, uitbreidbaarheid, en prestaties worden verbeterd zonder bestaande functionaliteit te verliezen?

De beperkingen van de oorspronkelijke Ricgraph Explorer lagen voornamelijk in de monolithische architectuur, waarin frontend- en backendfunctionaliteit sterk met elkaar verweven waren binnen dezelfde Flask-applicatie, en in het ontbreken van paginering op API-endpoints die grote datasets retourneerden. Beide knelpunten zijn in dit project aangepakt: de applicatie is opgesplitst in een zelfstandige React/TypeScript-frontend en een Flask-backend die uitsluitend via een REST API met elkaar communiceren, en de relevante API-endpoints zijn voorzien van cursor-based paginering.

Voor de frontend/backend-scheiding is gekozen voor een architectuur waarin de frontend is opgebouwd uit modulaire, herbruikbare componenten, met name zichtbaar in de generieke tabel- en pagineringscomponenten. Dit sluit aan bij de doelstelling om de applicatie beter onderhoudbaar en uitbreidbaar te maken.

De vier deelvragen zijn in hoofdstuk 6 beantwoord. Samengevat: de beperkingen van de oorspronkelijke architectuur zaten in de server-side gegenereerde interface die verweven was met de data laag en in het ontbreken van paginering (deelvraag 1). Het opsplitsen vereiste een volledige REST-API met een expliciet contract en een zelfstandig herbouwde frontend (deelvraag 2). Cursor-based paginering is de meest geschikte vorm voor de grote, dynamische datasets van Ricgraph, met de kanttekening dat niet elke endpoint cursor-pagineerbaar is (deelvraag 3). Of de nieuwe oplossing een aantoonbare verbetering vormt, is kwalitatief onderbouwd maar niet met metingen, een meetopzet is aangereikt (deelvraag 4).

Voor de paginering is eerst een eigen implementatie gebouwd en later de door de opdrachtgever aangeleverde implementatie overgenomen, zodat de frontend aansluit op het contract dat binnen Ricgraph wordt onderhouden. Beide benaderingen halen de resultaten set eenmalig op en cachen deze. Een volwaardige databasecursor zou naar verwachting efficiënter zijn, maar bleek binnen de projecttijd niet haalbaar en blijft een punt voor vervolgonderzoek.

Samenvattend is de architectuur gemoderniseerd op een manier die de onderhoudbaarheid en uitbreidbaarheid van de applicatie verbetert. Bestaande functionaliteit is daarbij niet verloren gegaan, omdat de oorspronkelijke interface naast de nieuwe frontend blijft bestaan. De nieuwe frontend zelf biedt het grootste deel van die functionaliteit, maar nog niet alles (zie 6.1). Twee onderdelen functioneren bovendien op een grote dataset niet goed (zie 6.2). Dat is kenmerkend voor een geleidelijke modernisering (zie 4.2): de oorspronkelijke interface kan pas worden uitgefaseerd wanneer de resterende functies beschikbaar zijn. De mate waarin dit ook tot een meetbare prestatieverbetering heeft geleid, blijft echter onderwerp van vervolgonderzoek.

8. Aanbevelingen

Hoewel de doelstellingen van dit project grotendeels zijn gerealiseerd, zijn tijdens de ontwikkeling verschillende mogelijkheden voor verdere verbetering naar voren gekomen.

Een belangrijke aanbeveling is het onderzoeken van de mogelijkheden voor het toepassen van database cursors binnen Neo4j. Tijdens dit project is cursor-based paginering gerealiseerd in de backend van Ricgraph Explorer. Oorspronkelijk was het de bedoeling om gebruik te maken van database cursors, waarbij de database zelf de positie binnen de dataset bijhoudt. Vanwege de complexiteit van de bestaande architectuur en de beschikbare projecttijd is uiteindelijk gekozen voor een implementatie in de backend. Een database cursor zou in theorie efficiënter kunnen zijn, omdat de database de resultaten direct vanaf de juiste positie kan ophalen zonder dat aanvullende verwerking in de backend nodig is. Daarom wordt aanbevolen om in de toekomst te onderzoeken of database cursors technisch haalbaar zijn en daadwerkelijk een prestatieverbetering opleveren ten opzichte van de huidige implementatie.

Ook wordt aanbevolen om de prestaties van de huidige implementatie kwantitatief te meten. Binnen dit project zijn de architectuur- en pagineringskeuzes onderbouwd op basis van kwalitatieve analyse, maar is niet gemeten of de gerealiseerde oplossing daadwerkelijk sneller of efficiënter is dan de oorspronkelijke, niet-gepagineerde situatie. Aanbevolen wordt om hiervoor concrete metingen uit te voeren, bijvoorbeeld van responstijden en geheugengebruik bij verschillende datasetgroottes, zodat toekomstige keuzes op dit gebied beter onderbouwd kunnen worden.

Bij het meten van de prestaties verdient de aanbeveling een concrete opzet te hanteren: meet responstijd en geheugengebruik van een vaste set representatieve verzoeken bij oplopende datasetgroottes, zowel op de oorspronkelijke, niet-gepagineerde situatie als op de huidige implementatie, en vergelijk daarnaast het aantal database-aanroepen per opgevraagde pagina.

Verder wordt aanbevolen de in 6.2 gevonden issues op te lossen. De aggregatie op de Open Science-profiel- en dashboardpagina's zou de volledige resultaten set moeten doorlopen in plaats van af te knippen op 250 resultaten. Daarnaast kan de voorbeeldtekst op de samenwerkingspagina worden aangepast aan de eigen installatie. Voor overige bekende tekortkomingen, met per punt een aanbeveling, zie bijlage 4.

Ook wordt aanbevolen de nieuwe frontend te voltooien, zodat de oorspronkelijke interface kan worden uitgefaseerd. Een deel van de ontbrekende functies kan direct worden gebouwd, omdat de API ze al ondersteunt. Voor de overige moet eerst de REST API worden uitgebreid.

Daarnaast wordt aanbevolen om geautomatiseerde tests toe te voegen. Op dit moment zijn die er niet. Tijdens dit project lag de nadruk voornamelijk op de realisatie van nieuwe functionaliteit en het uitvoeren van functionele controles tijdens de

ontwikkeling. Door geautomatiseerde tests op te zetten kunnen toekomstige wijzigingen sneller worden gevalideerd en kunnen regressies eerder worden opgespoord.

Verder kan worden onderzocht of aanvullende optimalisaties mogelijk zijn voor het verwerken van grote datasets. Hierbij kan worden gedacht aan verdere optimalisatie van databasequery's, cachingstrategieën, of technieken zoals streaming of server-side chunking wanneer datasets in de toekomst verder groeien.

Daarnaast wordt aanbevolen om de bruikbaarheid van de nieuwe frontend te toetsen bij daadwerkelijke gebruikers. Binnen dit project is de bruikbaarheid niet los getest, bijvoorbeeld door middel van gebruikerstests met onderzoekers die Ricgraph Explorer in de praktijk gebruiken. Dergelijke tests kunnen inzicht geven in eventuele verbeterpunten die niet uit functionele controles alleen naar voren komen.

Ten slotte wordt aanbevolen de technische documentatie verder uit te breiden. Hoewel tijdens dit project aanvullende documentatie en codecommentaar zijn toegevoegd, kan een uitgebreidere beschrijving van de architectuur, componentstructuur, en API toekomstige ontwikkelaars ondersteunen bij het onderhouden en verder ontwikkelen van Ricgraph Explorer.

9. Evaluatie

9.1 Evaluatie van de procesgang

De samenwerking met de bedrijfsbegeleider verliep via de in hoofdstuk 4 beschreven wekelijkse voortgangsgesprekken en tweewekelijkse sprintreviews. Tot het moment van oplevering hebben twaalf sprintreviews plaatsgevonden. Eén geplande review is komen te vervallen wegens persoonlijke omstandigheden. Deze aanpak zorgde ervoor dat bevindingen, zoals de technische discussie over de efficiëntie van de cursor-implementatie, snel konden worden meegenomen in de verdere ontwikkeling.

Van 16 juli tot 1 september was de bedrijfsbegeleider niet bereikbaar. In die periode is doorgewerkt aan de frontend, hoewel de voortgang lager lag dan gepland. De overname van de nieuwe pagineringsimplementatie heeft na die periode plaatsgevonden.

Achteraf gezien was de initiële inschatting van de benodigde tijd voor het project te optimistisch. Met name de mate waarin de frontend nog geherstructureerd moest worden om voldoende modulair te zijn, en de tijd die nodig was om alle resultaatpagina's af te ronden, werden bij aanvang van het project onderschat. Dit werd gedurende het traject steeds duidelijker en heeft uiteindelijk geleid tot de beslissing om de stage met één periode te verlengen. Een realistischere inschatting van de complexiteit van de bestaande Ricgraph Explorer-codebase, mogelijk op basis van een uitgebreidere verkenning voorafgaand aan de planning, had dit wellicht kunnen voorkomen.

Ook het backendwerk aan de cursor-paginerings is bij aanvang onderschat. Het begrijpen van de bestaande backend-code, het onderzoeken van een databasecursor, en het bouwen van een eigen implementatie op negen endpoints inclusief de cachinglaag, kostten samen zeker zoveel tijd als het frontendwerk en vormden daarmee een even grote oorzaak van de uitloop.

Het grotendeels laten genereren van de code voor de drie meest complexe pagina's was een afweging onder tijdsdruk. Het leverde op korte termijn snelheid op, maar juist deze pagina's zijn de pagina's waarvan de werking en de juistheid van de getoonde gegevens nu niet met zekerheid kunnen worden vastgesteld (zie 6.2). Achteraf had een kleiner, wel volledig doorgrond deel van deze functionaliteit een bruikbaar resultaat opgeleverd dan drie volledige pagina's waarvan de werking niet te verantwoorden is.

9.2 Planning, risico's, en afwijkingen

De planning van dit project is gedurende het traject op een aantal punten bijgesteld ten opzichte van de oorspronkelijke opzet.

In kalenderweek 27 is in overleg met de docentbegeleider besloten de stage met één periode te verlengen. De belangrijkste reden hiervoor was de inschatting dat de

resterende werkzaamheden niet met voldoende zekerheid binnen de oorspronkelijke planning afgerond konden worden, mede gezien de aankomende zomervakantie waarin een verminderde voortgang te verwachten was. De verlenging bood meer ruimte om het project op een verantwoorde manier af te ronden, zonder de druk van een te krappe planning.

Een risico dat vooraf is onderkend en in het plan van aanpak is opgenomen, betrof de beperkte beschikbaarheid van stakeholders tijdens de zomerperiode: de bedrijfsbegeleider was van 16 juli tot 1 september niet bereikbaar, en ook de examinerator was voor een deel van deze periode afwezig. Dit risico heeft zich deels gerealiseerd: in de kalenderweken die in de vakantie vielen is minder voortgang geboekt, als gevolg van de zomervakantie in combinatie met persoonlijke omstandigheden.

Naast de aanpassingen in de planning heeft één inhoudelijke koerswijziging plaatsgevonden: de eigen implementatie van cursor-based paginering is vervangen door die van de bedrijfsbegeleider. De afweging daarachter is beschreven in hoofdstuk 5. Voor de planning betekende dit dat de backend moest worden teruggezet en de frontend opnieuw op het cursor-contract moest worden aangesloten, in de laatste projectfase.

Twee vooraf onderkende risico's hebben zich daadwerkelijk voorgedaan. Het risico dat cursor-based paginering niet op alle zoekfunctionaliteiten toepasbaar zou zijn, trad op bij de endpoint die twee onafhankelijke lijsten in één response retourneert. Dit is opgevangen door die endpoint zonder cursor te benaderen, met een begreemd maximum-aantal resultaten en zonder paginering in de interface. Het risico van beperkte beschikbaarheid van stakeholders tijdens de zomer heeft geleid tot minder voortgang in de vakantieweken. Dit is deels opgevangen door voor de vakantie te bespreken wat er moest gebeuren, maar de voortgang bleef in deze weken achter bij de planning.

9.3 Ethische afwegingen

Binnen dit project zijn de ethische aspecten beperkt, aangezien de opdracht zich voornamelijk richt op de technische modernisering van een bestaande applicatie. Desondanks is tijdens de uitvoering rekening gehouden met een aantal relevante aandachtspunten.

Ricgraph verwerkt metadata over onderzoekers, organisaties, en onderzoeksresultaten. Tijdens dit project zijn deze gegevens uitsluitend geraadpleegd voor de ontwikkeling en het testen van de applicatie. Er zijn geen gegevens toegevoegd, gewijzigd, of verwijderd. Daarnaast waren de gebruikte databronnen afkomstig uit publieke bronnen, waardoor geen vertrouwelijke of afgeschermd onderzoekgegevens zijn verwerkt.

Een belangrijk aandachtspunt was het behouden van de betrouwbaarheid van de applicatie. De nieuwe frontend en de aanpassingen aan de backend mochten geen invloed hebben op de inhoud van de onderzoeksgegevens of de juistheid van de

weergegeven informatie. Daarom is de opdracht beperkt gebleven tot het verbeteren van de architectuur, de gebruikersinterface, en de manier waarop gegevens worden opgevraagd en weergegeven.

Omdat Ricgraph een open-source project is, is tijdens de ontwikkeling ook aandacht besteed aan de overdraagbaarheid van de gerealiseerde wijzigingen. De code is voorzien van aanvullende documentatie en commentaar waar dit de leesbaarheid ten goede komt, zodat toekomstige ontwikkelaars de aangebrachte wijzigingen eenvoudiger kunnen begrijpen en onderhouden.

9.4 Randvoorwaarden en kwaliteitsbewaking

De in het plan van aanpak genoemde randvoorwaarden zijn gedurende het project vervuld: er was toegang tot de Ricgraph-codebase en tot testdatasets, er was regelmatige afstemming met de bedrijfsbegeleider (met uitzondering van de vakantieperiode, zie hierboven), en de bestaande documentatie van Ricgraph was beschikbaar.

De kwaliteit van het werk is op meerdere manieren bewaakt: via overleggen en codebesprekingen beschreven in hoofdstuk 4 en 9.1, doorlopende functionele controle tijdens de ontwikkeling (zie 4.4), handmatige controle van de frontend bij elke wijziging, en statische codeanalyse (linting). De eindcontrole van de volledige applicatie staat in hoofdstuk 6 ("Validatie en testresultaten").

10. Verwijzingen

- Alternatives, Inspiration and Comparisons.* (sd). Opgehaald van FastAPI: <https://fastapi.tiangolo.com/alternatives/#flask>
- Anthropic. (2026). Opgehaald van Claude: <https://claude.ai>
- Awati, R., & Wigmore, I. (2024, 10 16). *What is monolithic architecture in software?* Opgehaald van TechTarget: <https://www.techtarget.com/whatis/definition/monolithic-architecture>
- Back4app Engineering. (2026, 07 23). *What is Decoupled Architecture (Headless Backend)?* Opgehaald van back4app: <https://www.back4app.com/glossary/decoupled-architecture/>
- Bootstrap.* (sd). Opgehaald van Bootstrap: <https://getbootstrap.com>
- Claritee. (2025, 05 27). *Material UI vs. Bootstrap: Selecting the Best UI Framework for Your Project.* Opgehaald van Claritee Blog: <https://claritee.io/blog/material-ui-vs-bootstrap-selecting-the-best-ui-framework-for-your-project/>
- Component Based Software Engineering.* (2022, 06 21). Opgehaald van GeeksforGeeks: <https://www.geeksforgeeks.org/software-engineering/component-based-software-engineering/>
- FastAPI.* (sd). Opgehaald van FastAPI: <https://fastapi.tiangolo.com/>
- Fateh, D. (2025, 04 09). *The ultimate guide to CSS frameworks in 2025.* Opgehaald van contentful: <https://www.contentful.com/blog/css-frameworks/>
- Flask.* (sd). Opgehaald van Flask: <https://flask.palletsprojects.com/en/stable/>
- Flask vs. FastAPI: Which One to Choose.* (2025, 07 23). Opgehaald van GeeksForGeeks: <https://www.geeksforgeeks.org/blogs/flask-vs-fastapi/>
- Foundation.* (sd). Opgehaald van Foundation Framework: <https://get.foundation/index.html>
- Fowler, M. (2015, 06 03). *Monolith First.* Opgehaald van martinFowler: <https://martinfowler.com/bliki/MonolithFirst.html>
- Fowler, M. (2024, 08 22). *Strangler Fig.* Opgehaald van martinFowler: <https://martinfowler.com/bliki/StranglerFigApplication.html>
- Gupta, L. (2025, 4 1). *What is REST?* Opgehaald van REST API Tutorial: <https://restfulapi.net/>
- Hartemink, V. (2026, 01 06). Onderzoek Ricgraph REST API pagination.
- ISO/IEC 25010.* (2024). Opgehaald van ISO 25000: <https://iso25000.com/index.php/en/iso-25000-standards/iso-25010>

- JavaScript Flavors*. (2020). Opgehaald van State of JS: <https://2020.stateofjs.com/en-US/technologies/javascript-flavors/>
- QuarkAndCode. (2025, 12 9). *API Pagination Explained: Techniques, Best Practices & Real-World Tips*. Opgehaald van Medium: <https://freedium-mirror.cfd/https://medium.com/@QuarkAndCode/api-pagination-explained-techniques-best-practices-real-world-tips-825ff43e3088>
- Rajput, N. (2025, 06 27). *I Switched from Flask to FastAPI. Here's What I Loved (and What I Didn't)*. Opgehaald van Medium: <https://medium.com/@hadiyolworld007/i-switched-from-flask-to-fastapi-heres-what-i-loved-and-what-i-didn-t-4f5bd66dd219>
- React*. (sd). Opgehaald van React: <https://react.dev/>
- TypeScript*. (sd). Opgehaald van TypeScript: <https://www.typescriptlang.org/>
- Ulili, S. (2025, 03 05). *Flask vs FastAPI: An In-Depth Framework Comparison*. Opgehaald van Better Stack: <https://betterstack.com/community/guides/scaling-python/flask-vs-fastapi/>
- Usage statistics of JavaScript as client-side programming language on websites*. (sd). Opgehaald van W3Techs: <https://w3techs.com/technologies/details/cp-javascript>
- Web Framework Benchmarks*. (2022, 07 19). Opgehaald van TechEmpower: <https://www.techempower.com/benchmarks/#section=data-r21&test=fortune>

11. Bijlages

11.1 Bijlage 1 – TypeScript vs JavaScript

TypeScript vs JavaScript

JavaScript

JavaScript is een programmeertaal die voornamelijk wordt gebruikt voor websites. In tegenstelling tot HTML en CSS — die statisch zijn en alleen de structuur en opmaak van een webpagina bepalen — maakt JavaScript het mogelijk om logica uit te voeren in de browser. Hierdoor kan een website interactief en dynamisch worden, bijvoorbeeld door het reageren op gebruikersinvoer of het ophalen van data zonder de pagina te verversen.

JavaScript bestaat al sinds 1995 en is in de loop der jaren uitgegroeid tot een onmisbaar onderdeel van het web. Tegenwoordig (2025) wordt JavaScript gebruikt door ongeveer 98,9% van alle websites. (Usage statistics of JavaScript as client-side programming language on websites, sd) Het moderne internet is dan ook nauwelijks denkbaar zonder JavaScript; vrijwel iedere grote website maakt er gebruik van.

Naarmate JavaScript intensiever en op grotere schaal werd ingezet, met name binnen grotere projecten en bedrijven, werden de beperkingen van de taal steeds duidelijker. Denk hierbij aan het ontbreken van strikte types en de grotere kans op fouten die pas tijdens runtime zichtbaar worden. Deze problemen vormden mede de aanleiding voor het ontstaan van TypeScript.

TypeScript

TypeScript werd voor het eerst uitgebracht in 2012 (versie 0.8) en is ontwikkeld door Microsoft. Het doel van TypeScript was om een veiligere en beter schaalbare variant van JavaScript te bieden, met name voor grote en complexe applicaties.

In de basis is TypeScript een superset van JavaScript. Dit betekent dat alle geldige JavaScript-code ook geldige TypeScript-code is. Tijdens het compileren wordt TypeScript omgezet naar gewone JavaScript, zodat het in elke browser kan worden uitgevoerd. Ontwikkelaars kunnen er bovendien voor kiezen om TypeScript-features wel of niet te gebruiken en eventueel grotendeels “standaard” JavaScript te schrijven binnen een TypeScript-project. Hierdoor is de overstap naar TypeScript laagdrempelig en vormt het geen groot risico bij projectkeuzes.

Een van de belangrijkste kenmerken van TypeScript is de toevoeging van statische types. Hierdoor kunnen ontwikkelaars expliciet aangeven welk datatype een variabele, functieparameter, of returnwaarde heeft. Dit zorgt voor betere foutdetectie

tijdens het ontwikkelen, duidelijkere code, en betere ondersteuning door editors en IDE's, zoals automatische aanvulling en refactoring.

(TypeScript, sd)

Uit onderzoek van State of JS (2020) blijkt dat TypeScript zeer positief wordt ontvangen binnen de ontwikkelaarsgemeenschap:

- 78% van de ontwikkelaars gebruikte TypeScript en zou het opnieuw/blijven gebruiken
- 70% gaf aan interesse te hebben en TypeScript te willen leren
- 93% was tevreden met TypeScript en zou het opnieuw/blijven gebruiken

(JavaScript Flavors, 2020)

Conclusie

Door de toevoeging van statische types, betere onderhoudbaarheid, en verbeterde ontwikkelervaring is TypeScript bij uitstek geschikt voor grotere en complexere projecten. Omdat TypeScript volledig compatibel is met JavaScript en geleidelijk kan worden ingevoerd, biedt het de voordelen van extra veiligheid zonder de flexibiliteit van JavaScript te verliezen.

Daarom is TypeScript een logische en aanbevolen keuze voor dit project.

React Bootstrap

Tiago Bastos

02-2026

React

React is een JavaScript library die wordt gebruikt voor het bouwen van gebruikersinterfaces op basis van componenten. Met React kan de structuur (het “skelet”) van een webapplicatie worden opgebouwd door herbruikbare componenten te combineren. React richt zich hierbij voornamelijk op logica en state-management, en niet op vormgeving.

De standaard HTML-elementen die in React worden gebruikt, hebben van zichzelf weinig visuele aantrekkingskracht. Om een professionele en consistente gebruikersinterface te realiseren, wordt React in de praktijk vrijwel altijd gecombineerd met een CSS-framework of UI-library. Deze libraries bieden vooraf gedefinieerde stijlen en/of componenten die het ontwikkelen van een aantrekkelijke interface versnellen.

De opdrachtgever heeft aanbevolen om React Bootstrap te gebruiken. In dit hoofdstuk wordt onderzocht wat React Bootstrap precies is, waarom het gebruikt kan worden, en welke alternatieven er bestaan.

Bootstrap

Om React Bootstrap goed te kunnen begrijpen, is het belangrijk om eerst Bootstrap zelf te beschrijven.

Bootstrap is één van de meest gebruikte CSS-frameworks ter wereld en wordt op grote schaal toegepast bij het ontwikkelen van websites en webapplicaties. (Bootstrap, n.d.) Het framework biedt ontwikkelaars vooraf ontworpen stijlen en componenten, zoals knoppen, navigatiebalken, formulieren, en modals.

Waar een ontwikkelaar in traditionele webontwikkeling zelf HTML schrijft, CSS toevoegt voor styling, en JavaScript toevoegt voor interactie, neemt Bootstrap een groot deel van dit werk over. Door simpelweg bepaalde CSS-classes toe te voegen aan HTML-elementen, worden deze automatisch voorzien van styling en soms ook interactieve functionaliteit.

Dit zorgt voor:

- Snellere ontwikkeling, omdat minder handmatige styling nodig is
- Consistentie in design, doordat dezelfde componenten herhaaldelijk worden gebruikt

- Lagere instapdrempel, vooral voor beginnende ontwikkelaars

Een nadeel van Bootstrap is dat het ontwerp relatief herkenbaar en minder flexibel is. Websites die sterk leunen op standaard Bootstrap-componenten kunnen visueel op elkaar gaan lijken. Dit is een veelgenoemd kritiekpunt en een reden waarom sommige ontwikkelaars alternatieven kiezen.

React Bootstrap

React Bootstrap is een library die React en Bootstrap met elkaar combineert. Het is geen simpele koppeling, maar een herimplementatie van Bootstrap-componenten als echte React-componenten.

Wanneer React en Bootstrap los van elkaar worden gebruikt, wordt Bootstrap geïnstalleerd als CSS- en JavaScript-library. Vervolgens worden Bootstrap-classes direct toegepast op HTML-elementen binnen React-componenten. Hoewel dit technisch werkt, sluit deze aanpak niet volledig aan bij de React-filosofie.

React Bootstrap lost dit probleem op door de JavaScript-functionaliteit van Bootstrap volledig te vervangen. Alle componenten zijn herschreven als React-componenten en maken geen gebruik van externe dependencies zoals jQuery, die standaard bij Bootstrap horen. (React, n.d.)

Hierdoor kunnen componenten worden gebruikt op een manier die natuurlijk aanvoelt binnen React, bijvoorbeeld via props en state. In plaats van HTML-elementen met Bootstrap-classes te schrijven, kunnen ontwikkelaars direct gebruikmaken van componenten zoals `<Button>`, `<Modal>`, `<Form>`, etc.

Voordelen van React Bootstrap zijn onder andere:

- Betere integratie met React
- Geen afhankelijkheid van jQuery
- Duidelijkere en leesbare component-structuur
- Behoud van de bekende Bootstrap-stijl

Alternatieven

Er zijn relatief weinig libraries die exact hetzelfde doel hebben als React Bootstrap. Wel bestaan er meerdere frameworks en libraries die een vergelijkbare rol vervullen binnen React-projecten.

Tailwind

Tailwind CSS is momenteel één van de meest populaire CSS-frameworks. In tegenstelling tot Bootstrap is Tailwind een zogeheten *utility-first framework*. Dit betekent

dat het geen kant-en-klare componenten levert, maar een uitgebreide set kleine CSS-classes waarmee ontwikkelaars zelf hun styling opbouwen.

Deze aanpak biedt veel flexibiliteit en controle over het ontwerp. Elementen kunnen zeer precies worden gestyled zonder eigen CSS-bestanden te schrijven. Een nadeel is dat HTML- en JSX-code hierdoor minder overzichtelijk kan worden door het grote aantal classes.

Tailwind wordt vaak gekozen door ontwikkelaars die maximale controle over hun design willen en minder waarde hechten aan standaard componenten.

Material UI

Material UI (MUI) is een component-gebaseerde UI-library die is gebaseerd op Google's Material Design-richtlijnen. Het framework biedt een zeer uitgebreide set moderne en visueel aantrekkelijke componenten en heeft een grote en actieve community. (Claritee, 2025)

Material UI is vooral populair in grotere en complexere applicaties. Een nadeel is dat het ontwerp sterk vastligt binnen de Material Design-stijl, wat minder geschikt kan zijn wanneer een uniek of afwijkend design gewenst is.

Foundation Framework

Foundation is, net als Bootstrap, een traditioneel CSS-framework waarbij componenten worden opgebouwd met HTML en gestyled via classes. (Foundation, n.d.) Het framework biedt minder componenten dan Bootstrap en wordt minder actief ontwikkeld.

Daarnaast worden de standaard stijlen van Foundation vaak als verouderd ervaren in vergelijking met modernere frameworks, waardoor het minder aantrekkelijk is voor nieuwe projecten.

Chakra UI

Chakra UI is een React-specifieke component library die zich richt op eenvoud, toegankelijkheid, en ontwikkelaarsvriendelijkheid. Het biedt vooraf gemaakte React-componenten die eenvoudig aanpasbaar zijn via props, zonder dat directe CSS-classes nodig zijn. (Fateh, 2025)

Chakra UI combineert het gemak van componenten met relatief veel flexibiliteit, maar wijkt qua stijl en aanpak duidelijk af van Bootstrap.

Conclusie

Op basis van dit onderzoek kan worden geconcludeerd dat de keuze van de opdrachtgever om React Bootstrap te gebruiken een verdedigbare en logische keuze is. React Bootstrap sluit goed aan bij React-projecten, vermindert complexiteit door het vermijden van jQuery en maakt het mogelijk om snel een consistente gebruikers-interface te ontwikkelen.

Hoewel alternatieven zoals Tailwind CSS en Material UI meer flexibiliteit of modernere ontwerpen bieden, brengen deze vaak ook een steilere leercurve of een sterkere ontwerpfilosofie met zich mee. React Bootstrap vormt daarmee een goede balans tussen gebruiksgemak, consistentie, en integratie met React.

11.3 Bijlage 3 – Flask vs FastAPI

Opmerking bij deze bijlage: Dit onderzoek is uitgevoerd in de eerste paar weken van het project en is daarna niet inhoudelijk herzien. Eén uitspraak in de vergelijking blijkt achteraf onjuist: sinds Flask 2.0 ondersteunt Flask zelf async/await in views, zodat asynchrone ondersteuning niet uitsluitend via externe oplossingen mogelijk is. Wel geldt dat elk verzoek een volledige worker blijft bezetten en dat deze opzet minder performant is dan die van een async-first framework (Flask, sd). De conclusie van de vergelijking verandert hierdoor niet.

Flask vs FastAPI

Tiago Bastos

02-2026

Inleiding

In dit document wordt onderzocht wat Flask en FastAPI zijn en waarin deze Python web frameworks van elkaar verschillen. Daarnaast worden de voor- en nadelen van beide frameworks vergeleken. Op basis van deze vergelijking wordt beoordeeld welk framework het meest geschikt is voor het project Ricgraph.

Flask werd geïntroduceerd in 2010 en groeide snel in populariteit door zijn minimalistische en uitbreidbare ontwerp. FastAPI verscheen later, in 2018, en speelde in op de opkomst van moderne Python-functionaliteiten zoals type-annotaties en asynchrone programmering. Hierdoor positioneert FastAPI zich nadrukkelijk als een modern alternatief binnen het Python web-ecosysteem, terwijl Flask wordt gezien als een meer volwassen en gevestigde oplossing.

Flask

Flask is een lichtgewicht micro web framework voor Python. Het framework is ontworpen om webontwikkeling eenvoudig en toegankelijk te maken, met name voor kleinere projecten en beginnende ontwikkelaars. Door het minimalistische karakter biedt Flask veel vrijheid: het bevat alleen de basisfunctionaliteiten zoals routing en request handling, zonder verplichte externe libraries of frameworks. Hierdoor kan het framework eenvoudig worden uitgebreid naar complexere applicaties wanneer dat nodig is.

(Flask, sd)

(Flask vs. FastAPI: Which One to Choose, 2025)

FastAPI

FastAPI is een modern en performant web framework dat specifiek is ontwikkeld voor het bouwen van API's met Python. Het framework richt zich voornamelijk op het ontwikkelen van RESTful API's en maakt intensief gebruik van moderne Python-functionaliteiten zoals type-annotaties. FastAPI beschikt over ingebouwde mechanismen voor onder andere inputvalidatie, wat bijdraagt aan de veiligheid en betrouwbaarheid van applicaties. Daarnaast is FastAPI ontworpen met schaalbaarheid en hoge performance in gedachten. De prestaties zijn vergelijkbaar met die van frameworks in talen zoals NodeJS en Go.

(FastAPI, sd)

(Flask vs. FastAPI: Which One to Choose, 2025)

Volgens de documentatie van FastAPI is het framework ook deels geïnspireerd door Flask. Net als Flask positioneert FastAPI zich als een micro-framework: een relatief eenvoudig en minimalistisch framework zonder overbodige functionaliteit. Dit maakt het voor ontwikkelaars mogelijk om zelf te bepalen welke extra componenten worden toegevoegd. Een dergelijk "kaal" ontwerp kan leiden tot betere prestaties en meer flexibiliteit bij het combineren van verschillende technologieën. Daarnaast heeft FastAPI het eenvoudige en overzichtelijke routing-systeem van Flask als inspiratie gebruikt.

(Alternatives, Inspiration and Comparisons, sd)

Uit een benchmarkonderzoek uit 2022 van een onafhankelijke derde partij blijkt dat FastAPI in alle geteste categorieën beter presteert dan Flask.

(Web Framework Benchmarks, 2022)

Verschillen

Het artikel van GeeksForGeeks (Flask vs. FastAPI: Which One to Choose, 2025) heeft wat voorbeelden van verschillen. De informatie in dit hoofdstuk komt uit dat artikel.

Hoewel Flask en FastAPI beide Python web frameworks zijn, verschillen ze fundamenteel in ontwerpdoel, functionaliteit, en geschiktheid voor bepaalde toepassingen. Flask is oorspronkelijk ontwikkeld als een algemeen micro web framework, terwijl FastAPI vanaf de basis is ontworpen met het bouwen van API's als primair doel.

Een belangrijk verschil zit in data-validatie. Flask biedt standaard geen automatische validatie van inkomende data; dit moet door de developer zelf worden geïmplementeerd of met externe libraries worden opgelost. FastAPI daarentegen heeft ingebouwde data-validatie via type-annotaties en Pydantic-modellen. Hierdoor worden foutieve requests automatisch onderschept en teruggekoppeld met duidelijke foutmeldingen in JSON-formaat. Dit verkleint de kans op runtime-fouten en verhoogt de betrouwbaarheid van de API.

Ook op het gebied van asynchrone verwerking verschillen de frameworks sterk. Flask is van oorsprong synchroon (WSGI) en niet ontworpen voor het efficiënt afhandelen van veel gelijktijdige requests. Asynchrone ondersteuning is alleen mogelijk via externe oplossingen. FastAPI is daarentegen gebouwd op een asynchrone architectuur (ASGI), waardoor het zeer geschikt is voor I/O-intensieve toepassingen en real-time systemen. Dit maakt FastAPI beter schaalbaar bij hoge belasting. Meer over de asynchroniteit van Flask in een hoofdstuk verderop.

Wat betreft performance presteert FastAPI aantoonbaar beter bij toepassingen met veel gelijktijdige verbindingen of netwerk- en database-operaties. Flask blijft echter voldoende performant voor kleine tot middelgrote applicaties waarbij schaalbaarheid geen kritische factor is.

Een ander verschil is de documentatie-ondersteuning. Flask vereist handmatige documentatie of externe tools om API-documentatie te genereren. FastAPI genereert automatisch interactieve documentatie (zoals Swagger/OpenAPI), wat zowel ontwikkeling als testen aanzienlijk vereenvoudigt.

Ten slotte speelt community en volwassenheid een rol. Flask bestaat al langer en beschikt over een zeer grote community, uitgebreide documentatie, en veel beschikbare extensies. FastAPI is relatief jonger en heeft een kleinere, maar snelgroeiende community. Daar staat tegenover dat FastAPI beter aansluit bij moderne Python-ontwikkelpraktijken, zoals type-hints en dependency injection.

Naast technische verschillen speelt ook de ontwikkelervaring een belangrijke rol. Flask biedt veel vrijheid, maar vereist dat validatie, documentatie en foutafhandeling grotendeels handmatig worden ingericht. Dit kan bij grotere projecten leiden tot herhalende code en inconsistenties. FastAPI stimuleert een meer gestructureerde ontwikkelstijl door het gebruik van type-annotaties en data-modellen, waardoor validatie, documentatie, en editor-ondersteuning automatisch worden afgedwongen. Dit draagt bij aan betere onderhoudbaarheid van grotere codebases.

Samenvattend kan gesteld worden dat Flask vooral geschikt is voor eenvoudige webapplicaties en projecten waarbij flexibiliteit en eenvoud belangrijker zijn dan performance. FastAPI is daarentegen beter geschikt voor moderne, schaalbare, en performance-kritische API-gedreven systemen waar validatie, snelheid, en onderhoudbaarheid een grotere rol spelen.

(Flask vs. FastAPI: Which One to Choose, 2025)

(Ulili, 2025)

Migratie van Flask naar FastAPI

Naast het vergelijken van Flask en FastAPI op basis van functionaliteit en performance is het voor het project Ricgraph relevant om te onderzoeken hoe complex het is om een bestaande Flask API te migreren naar FastAPI. Een overstap moet niet alleen technisch haalbaar zijn, maar ook beheersbaar qua ontwikkeltijd en risico.

Uit praktijkervaringen van ontwikkelaars blijkt dat de overstap van Flask naar FastAPI over het algemeen goed te doen is, maar niet zonder aanpassingen. (Rajput, 2025) De basisconcepten van beide frameworks, zoals routing en request/response-afhandeling, lijken sterk op elkaar. Routes kunnen vaak één-op-één worden herschreven, waarbij decorators en endpoint-structuren herkenbaar blijven.

Een belangrijk verschil is echter dat FastAPI sterk leunt op type-annotaties en data-modellen. Waar Flask veelal werkt met dynamische dictionaries (`request.get_json()`), vereist FastAPI het expliciet definiëren van datamodellen met Pydantic. Dit betekent dat bestaande endpoints herschreven moeten worden om gebruik te maken van deze modellen. Hoewel dit initieel extra werk oplevert, resulteert het wel in schonere code, minder foutafhandeling, en betere onderhoudbaarheid.

Voordelen tijdens migratie

Volgens (Rajput, 2025) biedt FastAPI tijdens een migratieproces meerdere directe voordelen:

- Automatische inputvalidatie vervangt handmatige checks
- API-documentatie wordt automatisch gegenereerd
- Type-annotaties verbeteren editor-ondersteuning en leesbaarheid
- Minder boilerplate-code per endpoint

Voor een middelgrote Flask API betekent dit dat een migratie initieel tijd kost, maar op langere termijn leidt tot een robuustere en beter gestructureerde codebase.

Nadelen en aandachtspunten

De belangrijkste drempel bij migratie is de **leercurve**. Ontwikkelaars moeten vertrouwd raken met:

- Pydantic-modellen
- Type hints (Optional, Union, Annotated)

- Asynchrone programmeerconcepten (async / await)

Daarnaast kan debugging van asynchrone achtergrondtaken complexer zijn dan bij traditionele synchrone Flask-applicaties. (Rajput, 2025) Dit vraagt om discipline in logging en foutafhandeling.

Gunicorn en asynchroniteit in Flask

De opdrachtgever gaf aan dat de huidige Flask API gebruikmaakt van Gunicorn. Gunicorn is een serverprogramma dat wordt gebruikt om Flask-applicaties in productie te draaien. Het zorgt ervoor dat meerdere verzoeken tegelijk kunnen worden afgehandeld, wat belangrijk is voor de prestaties en stabiliteit van een API.

Door Gunicorn te gebruiken met meerdere zogeheten *workers* kan een Flask API meerdere gebruikers of verzoeken tegelijkertijd verwerken. Dit betekent echter niet automatisch dat de applicatie zelf volledig asynchroon werkt. Flask blijft in de basis een framework dat verzoeken één voor één afhandelt binnen een worker, ook al zijn er meerdere workers tegelijk actief.

Gunicorn biedt daarnaast een optie om verzoeken op een meer gelijktijdige manier af te handelen, wat de schaalbaarheid van een Flask API kan verbeteren bij hogere belasting. Deze oplossing kan in de praktijk voldoende zijn voor veel toepassingen en verklaart waarom een Flask API met Gunicorn toch goed kan presteren.

FastAPI is daarentegen vanaf de basis ontworpen met gelijktijdige verwerking in gedachten. Hierdoor kan het framework efficiënter omgaan met veel gelijktijdige verzoeken, zonder dat hiervoor extra configuratie of speciale serverinstellingen nodig zijn.

Samenvattend kan worden gesteld dat Gunicorn de prestaties van een Flask API aanzienlijk kan verbeteren, maar dat FastAPI structureel beter aansluit bij toepassingen waarbij schaalbaarheid en gelijktijdige verwerking een belangrijke rol spelen.

Conclusie

Op basis van dit onderzoek kan worden geconcludeerd dat zowel Flask als FastAPI geschikte Python web frameworks zijn, maar dat zij zich richten op verschillende gebruiksscenario's.

Flask is een volwassen, stabiel, en flexibel micro-framework dat zeer geschikt is voor eenvoudige webapplicaties en kleinere API's. In combinatie met Gunicorn kan Flask voldoende performant en schaalbaar zijn voor veel productieomgevingen. Wel vereist Flask meer handmatige implementatie van validatie, documentatie, en structuur, wat bij groeiende projecten kan leiden tot extra onderhoudswerk.

FastAPI positioneert zich als een modern API-framework en biedt duidelijke voordelen op het gebied van performance, automatische validatie, en onderhoudbaarheid. De ingebouwde documentatie en gestructureerde manier van werken sluiten goed aan bij moderne API-gedreven projecten zoals Ricgraph.

Daarbij moet worden opgemerkt dat de keuze voor FastAPI afhankelijk is van de bereidheid om een bestaande Flask API te migreren. Deze migratie kost tijd en vraagt aanpassingen in de codebase. Wanneer deze overstap past binnen de beschikbare tijd en prioriteiten van het project, is FastAPI de meest toekomstbestendige keuze. Indien dit niet het geval is, blijft Flask in combinatie met Gunicorn een valide en stabiele oplossing.

11.4 Bijlage 4 – Stand van zaken

Stand van zaken Ricgraph

In dit document zijn alle bevindingen uit het testen van de nieuwe Ricgraph-frontend opgenomen, met per punt een situatieschets en een aanbeveling. De bevindingen die van invloed zijn op de beoordeling van de kwaliteitscriteria zijn ook in 6.2 beschreven.

Timeout

Situatie

Er is een timeout van 10 seconden toegevoegd. Dit wil zeggen dat als een request naar de backend niet binnen die 10 seconden wordt beantwoord, gaat de applicatie er van uit dat de backend onbereikbaar is. Dit kan een probleem zijn als er requests zijn die langer dan die 10 seconden duren, bijvoorbeeld als de backend veel dingen moet ophalen of berekenen. Als dit een risico vormt, wordt er aanbevolen om een andere oplossing te nemen hiervoor.

Gerelateerd hieraan doet de BackendStatusBanner elke 5 seconden een refetch van alle queries die op dat moment in een error-status staan. Als de backend onbereikbaar is en er meerdere queries tegelijk falen, geeft elke tick een burst van parallelle requests, die zich blijft herhalen zolang de backend niet bereikbaar is.

Aanbeveling

Bijvoorbeeld: een aparte, lichte, endpoint maken in de backend, die alleen een heel simpel/kort antwoord geeft, gewoon zodat de frontend weet dat de backend aan staat. Of, zonder de backend te veranderen, kan er ook een bestaande endpoint gequeryd worden (zoals `get_ricgraph_info`), en vastleggen of dat antwoord geeft. Die kan zelfs een kortere timeout hebben, want we gaan er van uit dat die endpoint snel zou antwoorden.

Table export

Situatie

Elke tabel heeft een “export table to CSV file” knop. Deze houdt momenteel geen rekening met de pagination. Er is een default waarde voor het aantal rijen dat het export, en het export alleen de rijen die op dat moment zichtbaar zijn in de tabel.

Aanbeveling

Die logica veranderen zodat het alle rijen in de tabel gebruikt voor de export.

Load all cap

Situatie

In usePaginatedQuery.ts is er een constante `LOAD_ALL_PAGE_CAP` die op 200 staat. Deze waarde is arbitrair gekozen. Er wordt van uit gegaan dat een gebruiker niet meer dan $50 * 200 (= 10000)$, of $250 * 200 (= 50000)$ resultaten in een tabel wil/gaat zien. Deze cap is er voornamelijk voor het geval dat de backend faalt om de cursor te eindigen (tegen de frontend te zeggen dat er geen pagina meer is), of als het in een loop terecht komt, dan kan de frontend stoppen bij die cap, in plaats van eindeloos requests heen en weer te verwerken, terwijl de gebruiker geen idee heeft wat er gaande is en ook eindeloos zou zitten wachten.

Aanbeveling

Een knop maken om de request te annuleren, en/of een (visuele) waarschuwing dat het laden is gestopt bij de cap, zodat de gebruiker ergens kan klikken om het voort te zetten, in het geval dat de gevraagde dataset wél groter is dan de cap en de gebruiker dat allemaal wil laden.

Onzichtbare pagination effect

Situatie

Als je in een tabbed table bent, en je klikt Next, en het haalt de volgende batch van data op, is er een kans dat je merkt dat de paginanummers niet veranderen. Dit is geen bug, maar een effect van de tabbed table. De data wordt wel opgehaald, maar in dit geval werd er data opgehaald van een andere tab dan waar je bent, waardoor het lijkt alsof er niks is veranderd. Dus er is wel een tab bijgekomen of aangevuld met die nieuwe data, en het aantal en de histogrammen worden wel geupdate, je merkt het alleen niet omdat de data die je binnenkreeg niet van de tab hoorde die je geselecteerd hebt.

Dit, of iets soortgelijks, is ook het geval in de details_view met de filters (op bijv. name en category)

Aanbeveling

De aanbeveling hier hangt af van hoe je de user experience wil hebben. Je kan bijvoorbeeld blijven fetchen totdat er in de tab waar je bent wél iets verandert. Je kan ook een bericht/banner/alert laten zien waar je de gebruiker informeert dat wat er net geupdate is, niet bij de tab hoort waar je momenteel bent.

Efficientie requests

Situatie

Als je een search doet met bijv. `page_size 50`, en dan terug gaat, en dezelfde search doet, maar dan met bijv. `page_size 3`, dan wordt op beide momenten een hele nieuwe "lookup" gedaan, terwijl je in theorie de data al zou moeten hebben van de eerste search.

Dit komt omdat `page_size` deel uitmaakt van de cache key waarmee de resultaten van een search worden bijgehouden. Een wijziging in `page_size` geeft dus een andere cache key, die een hele nieuwe paginering begint, met een nieuwe request.

Aanbeveling

Frontend

Ontkoppel de `page_size` die naar de backend wordt gestuurd van de `page_size` die gebruikt wordt om resultaten weer te geven. Vraag de backend altijd om een vaste, ruimere hoeveelheid resultaten op, en laat het aantal zichtbare rijen per pagina volledig client-side bepalen op basis van de data die al in het geheugen zit. Een wijziging in de weergave-`page_size` vereist dan geen nieuwe request meer, omdat de onderliggende zoekopdracht (en dus de cache-key) niet meer verandert.

Backend

De bestaande batch-cache is gekoppeld aan een willekeurig gegenereerde sleutel per nieuw verzoek, niet aan de zoekopdracht zelf. Hierdoor wordt een cache-hit alleen bereikt binnen een lopende paginering, niet wanneer dezelfde zoekopdracht na verloop van tijd (of in een nieuwe sessie/tab) opnieuw wordt uitgevoerd. Dit zou verbeterd kunnen worden door de cache-sleutel af te leiden van de zoekopdracht zelf (functienaam + parameters), eventueel met een tijdslimiet, zodat ook een "koude" herhaling van dezelfde search de cache kan hergebruiken.

Paginering-knoppen verspringen

Situatie

Tabelrijen verschillen in hoogte, afhankelijk van hoe lang de inhoud van een cel is (bijvoorbeeld een lange comment of url). De pagination knoppen staan onder de tabel, dus hun positie op het scherm verschuift mee met de hoogte van de huidige pagina. Hierdoor moet je na elke klik opnieuw met de muis naar de knoppen zoeken, of zelfs scrollen, om door te kunnen naar de volgende pagina.

Aanbeveling

Plaats de paginering knoppen boven de tabel in plaats van eronder. Omdat de knoppen dan vóór de wisselende inhoud in de layout staan, verandert hun positie niet meer wanneer de tabelinhoud van hoogte verandert. Dit is een kleine aanpassing zonder weerslag op de rest van de layout.

Andere alternatieven: een vaste hoogte met interne scrollbar voor de tabel, of de knoppen "sticky" onderaan het scherm plakken, dus dat ze niet bewegen terwijl je scrollt, maar dit past niet goed bij pagina's met meerdere onafhankelijke tabellen tegelijk, zoals hier het geval is.

Leeg filter niet te onderscheiden van "geen filter"

Situatie

In `useFilterCheckboxes.ts` wordt de filterselectie bijgehouden als een lijst geselecteerde waarden in de URL. Als een gebruiker alle checkboxes uitzet, wordt die lijst leeg, en een lege lijst verdwijnt uit de URL (lege arrays worden simpelweg weggelaten). Na een her-navigatie of refresh is "alles bewust uitgevinkt" dan niet meer te onderscheiden van "nog nooit gefilterd", waardoor de UI alles weer aanvinkt.

Aanbeveling

Iets in de URL toevoegen om "expliciet leeg" te onderscheiden van "niet ingesteld", bijvoorbeeld een extra parameter of een waarde in de lijst zelf.

Verken samenwerkingen onbruikbaar op grote datasets

Situatie

Het endpoint voor samenwerkingen ondersteunt geen paginering en geeft de volledige resultaten set in één antwoord terug. Op een grote dataset duurt dat langer dan de timeout van tien seconden die de frontend hanteert, waarna de frontend de backend als onbereikbaar beschouwt en de statusbanner toont. De pagina is daardoor op zulke datasets niet bruikbaar.

Aanbeveling

Paginering toevoegen aan dit endpoint, of de aggregatie server-side uitvoeren zodat het antwoord klein blijft. Zolang dat niet is gebeurd, kan een ruimere timeout het probleem verzachten maar niet oplossen.

Aggregatie beperkt tot 250 resultaten

Situatie

De Open Science-profiel- en dashboardpagina's halen de resultaten van een organisatie op met een paginagrootte van 250 en volgen de cursor daarna niet verder. Organisaties met meer resultaten worden ondergerapporteerd. Zichtbaar in het dashboard voor Utrecht University: 203 datasets, 42 overige bijdragen, en 5 softwareproducten, samen exact 250. Tellingen wijken daardoor af van tellingen voor hetzelfde item elders in de applicatie.

Aanbeveling

De betreffende hook de volledige cursor laten doorlopen. Omdat dat bij grote organisaties veel verzoeken oplevert, is het verstandig dit te combineren met een voortgangsindicatie, of met een endpoint dat de aggregatie server-side uitvoert.

Voorbeeldtekst verwijst naar de demoserver

Situatie

De toelichting op de pagina "verken samenwerkingen" gebruikt organisatienamen van de publieke Ricgraph-demoserver, zoals "DUT". Deze tekst is ongewijzigd overgenomen uit de oorspronkelijke interface en past niet bij installaties met andere organisaties.

Aanbeveling

De voorbeelden per installatie configureerbaar maken, of vervangen door een uitleg zonder concrete organisatienamen.